

隆業工業大學

算法与程序设计

课程设计(论文)

中学生信息管理系统设计与实现

院（系）名称	电子与信息学院
专业班级	计算机类 23X
指导教师	待填写

姓名	学号

起 止 时 间： 2026.6.8-2026.6.12

课程设计分工表

组号	X	题目	基于 RBAC 的中学生信息管理系统设计与实现		
任务号	完成项目内容（作品、论文）详细介绍		完成人	作品模块	论文页码
1	本人主要负责中学生信息管理系统需求分析以及学生信息、年级管理、班级管理、课程基础资料模块的分析与论文编写工作。首先结合学校学生管理业务，对系统需求进行了整理，明确学生档案、年级、班级及课程等基础数据之间的业务关系，为后续系统设计提供依据。在学生管理模块中，重点分析学生信息的新增、查询、修改、删除以及分页展示等功能实现流程，并结合 StudentController 控制器梳理页面请求、业务处理和数据库访问之间的调用关系。随后完成 GradeController、ClazzController 和 CourseController 等模块的分析，对年级管理、班级管理和课程管理的业务流程进行了整理，说明各模块之间的数据关联及功能实现方式。同时结合论文内容，对需求分析、功能分析及基础资料管理等章节进行了完善，使论文描述与系统代码保持一致，并检查章节内容、功能流程及模块划分是否符合系统实际运行情况，为后续成绩管理、课程安排及统计报表等业务模块提供完整的基础数据支撑。		成员一（待填写）	第 1 章绪论； 2.2 功能需求分析； 学生信息管理、年级管理、班级管理、课程管理等基础资料模块	第 1 章、第 2 章（部分）、第 4 章（4.2.1 ~ 4.2.4）
2	本人主要负责数据库设计分析及数据库章节的论文编写工作。通过阅读系统 schema.sql、data.sql 初始化脚本以及 Mapper XML 映射文件，对学生、年级、班级、课程、成绩、教师、部门、考勤、学籍状态、角色、权限和课程安排等数据表进行了整理，分析各数据表之间的关联关系以及字段含义。结合数据库设计过程，对系统数据库概念设计、逻辑设计和实体关系进行了详细说明，并完成学生实体、课程实体、成绩实体、教师实体等实体分析内容。在数据库访问实现方面，对 MyBatis Mapper 映射、动态 SQL、关联查询及 PageHelper 分页查询实现方式进行了总结，同时分析数据库连接配置、数据初始化流程以及数据库与业务层之间的调用关系。论文编写过程中，完成数据库设计章节、实体关系图、数据表结构说明及数据库分析内容，保证数据库结构、论文描述与系统源代码保持一致，为系统各项业务功能提供稳定可靠的数据支持。		成员二（待填写）	数据库概念设计、数据库逻辑设计、实体关系图、各数据表结构、数据库实现	第 3 章、第 4 章 4.1
3	本人主要负责成绩管理、考勤管理、学籍状态管理及课程安排模块的分析与论文编写工作。结合系统源代码，对 ScoreController、AttendanceController、StudentStatusController 和 ScheduleController 等控制器进行了详细分析，整理各模块的业务流程、请求路径及数据处理过程。在成绩管理模块中，对成绩录入、查询、修改、删除以及课程关联等功能进行了分析，说明成绩数据与学生、课程之间的关系；在考勤管理模块中，对学生考勤记录维护、考勤状态管理及查询统计流程进行了整理；在学籍状态模块中，对学生入学、转学、毕业等状态变化管理进行了分析；在课程安排模块中，对班级、课程、教师、星期及节次之间的数据关联进行了总结。同时结合系统实现，对删除业务数据时关联数据处理进行了说明，保证系统数据一致性。论文编写过程中，完成相关章节内容，确保论文分析与系统功能实现一致。		成员三（待填写）	成绩管理、考勤管理、学籍状态管理、课程安排模块	第 4 章（4.2.5 ~ 4.2.8）
4	本人主要负责系统登录模块、角色权限基础信息管理以及统计报表模块的分析，并完成相关论文内容编写。首先结合 LoginController，对系统登录、退出及首页跳转流程进行了分析，整理用户登录验证、Session 保存及退出登录的处理过程。随后阅读 RoleController 和 PermissionController，对角色管理、权限管理等模块进行了分析，整理角色名称、角色编码、权限名称、访问路径等基础信息维护流程，并结合系统实际实现情况说明本项目主要完成角色和权限基础数据管理，不涉及完整 RBAC 权限控制框架。在统计报表模块中，对 ReportController 进行了分析，整理学生统计报表、成绩统计报表、教师统计报表、课程安排统计及考勤统计等功能的数据来源、统计过程及页面展示方式。论文编写过程中，完成登录模块、角色权限模块及统计报表模块相关章节，确保论文内容与系统实现保持一致，并对章节内容进行了统一整理和完善。		成员四（待填写）	登录模块、教师管理、部门管理、角色管理、权限管理、学生统计报表、成绩统计报表	第 2 章（登录分析）、第 4 章（4.2.9 ~ 4.2.14）
5	本人主要负责课程设计论文整体排版、系统测试及最终校对工作。按照学校课程设计论文模板，对论文标题、目录、章节编号、字体字号、段落格式、页眉页脚以及图表编号进行了统一调整，确保论文格式符合学校规范要求。同时对系统登录、学生管理、课程管理、成绩管理、考勤管理、课程安排及统计报表等主要功能进行了测试，记录各模块运行情况，并整理测试结果及相关截图，完善论文系统实现部分内容。在论文整理过程中，对图文对应关系、章节顺序、参考文献格式及数字、英文字符字体进行了统一检查，及时修改格式错误和模板残留内容，保证论文整体结构完整、内容规范、排版统一。同时对论文进行了全面校对，检查文字表述是否准确、模块描述是否与程序一致，确保最终提交版本满足课程设计论文的格式要求和内容要求。		成员五（待填写）	系统测试、论文排版、截图整理、格式规范、参考文献、总结、最终校对	第 5 章、参考文献及全文整理

课程设计（论文）任务及评语

院（系）：电子与信息学院

教研室：软件工程

组 号	X	题 目	计算机类 23X
课程设计（论文）任务	任务要求： 中学生信息管理系统主要实现学生管理、基础资料管理、教学业务管理、系统管理和统计报表五个功能模块。 （1）基础资料管理模块分为以下五个部分： 年级管理、班级管理、课程管理、教师管理、部门管理等。 （2）学生管理模块分为以下五个部分： 学生信息管理、学籍状态管理、学生信息查询、学生信息维护、学生档案管理等。 （3）教学业务管理模块分为以下五个部分： 成绩管理、考勤管理、课程安排管理、成绩查询、考勤查询等。 （4）系统管理模块分为以下五个部分： 用户登录、角色管理、权限管理、系统退出、基础信息维护等。 （5）统计报表模块分为以下五个部分： 学生统计报表、成绩统计报表、教师统计报表、考勤统计报表、课程安排统计报表等。 技术要求： 1. 用 E-R 图的设计方法设计数据库，采用 MySQL 作为数据库工具。 2. 数据库设计范式要达到 3NF。 3. 数据库与前台界面连接采用 JDBC 技术。 4. 软件能正常运行。软件界面要友好、操作要简单。 5. 采用 Java 语言，在 IDE 下编写代码，不少于 200 行。		
指导教师评语及成绩	作品成绩：_____ 答辩成绩：_____ 论文成绩：_____ 总成绩：_____ 指导教师签字： 年 月 日		

注：作品成绩占 20%，答辩成绩占 40%，论文成绩占 40%。

摘要

随着学校教学管理工作逐步向信息化、规范化方向发展，传统依赖纸质表格和人工统计的学生管理方式已经难以满足中学日常管理的效率要求。学生基本信息、年级班级、课程安排、成绩记录、考勤情况、学籍状态以及教师部门等数据之间存在较强关联，如果缺少统一的信息平台，容易出现数据重复录入、查询效率低、统计口径不一致和维护成本较高等问题。针对上述情况，本文结合课程设计源代码，围绕基于 Spring Boot 的中学生信息管理系统展开分析与设计说明。

系统采用 B/S 结构，后端以 Spring Boot 为基础整合 Spring MVC、MyBatis、PageHelper 和 Druid 连接池，数据库采用 MySQL，页面展示采用 Thymeleaf 模板并结合 AdminLTE 样式。系统主要实现登录、学生信息管理、年级管理、班级管理、课程管理、成绩管理、课程安排管理、教师部门管理、学籍状态管理、考勤管理、角色权限基础信息管理和统计报表等功能。各业务模块在控制层、服务层和持久层之间保持较清晰的分层关系，控制器负责接收页面请求并组织模型数据，服务层完成业务调度和关联数据处理，Mapper 层负责数据库访问。

本文在阅读系统代码和数据库脚本的基础上，对系统开发背景、需求分析、数据库概念设计、数据库逻辑设计以及主要功能实现进行了系统阐述。数据库部分围绕学生、年级、班级、课程、成绩、教师、部门、学籍状态、考勤、角色、权限和课程安排等实体展开，说明各表字段含义及实体间关系。系统实现部分则结合主要功能页面预留截图位置，分析各模块的业务流程和代码实现特点。通过本系统，可以提高中学生信息维护、查询和统计工作的效率，为学校日常教学管理提供较为完整的数据支撑。

关键词：中学生信息管理；Spring Boot；MyBatis；MySQL；权限管理

目 录

摘 要.....	i
第 1 章 绪论.....	1
1.1 系统开发背景.....	1
1.2 社会需求情况及设计的意义.....	1
1.3 系统达到的目标.....	1
1.4 系统开发工具及环境.....	2
1.4.1 Java 简介.....	2
1.4.2 MySQL 简介.....	2
第 2 章 系统分析设计.....	4
2.1 可行性分析.....	4
2.1.1 技术可行性.....	4
2.1.2 运行可行性.....	4
2.2 功能需求分析.....	4
第 3 章 数据库设计.....	7
3.1 数据库概念设计.....	7
3.2 数据库逻辑设计.....	11
第 4 章 系统实现.....	16
4.1 数据库实现.....	16
4.2 功能实现.....	16
第 5 章 总结.....	33
参考文献.....	34

第 1 章 绪论

1.1 系统开发背景

中学阶段的学生管理工作具有数据量持续增长、业务环节多、信息更新频繁等特点。学校在日常教学管理中需要维护学生基本档案、班级年级归属、课程安排、成绩记录、考勤情况、学籍变化以及教师部门等信息。若这些信息长期分散在纸质表格、电子表格或不同管理人员个人文件中，不仅会增加重复录入和人工核对的工作量，也会使数据查询、统计和追溯变得困难。

随着校园信息化建设的推进，使用 Web 系统承载教学管理数据已经成为较常见的解决方式。Web 系统不受单台计算机限制，管理人员可以通过浏览器访问统一入口，在同一套数据库中完成信息维护和查询。相较于传统方式，信息管理系统可以通过表结构约束、统一页面入口和权限基础数据维护减少人为差错，并通过分页查询、条件筛选和统计报表提升数据处理效率。

本课程设计所分析的项目是一个基于 Spring Boot 的中学生信息管理系统。系统代码采用典型的控制层、服务层、持久层和模板页面结构，围绕学校管理中的学生、年级、班级、课程、成绩、考勤、学籍状态、教师、部门、角色、权限和统计报表等内容展开。该项目虽然规模适中，但覆盖了 Web 后台管理系统中较完整的增删改查、分页查询、关联查询和页面渲染流程，适合作为课程设计论文的分析对象。

1.2 社会需求情况及设计的意义

在基础教育管理中，学生信息的准确性和可追溯性直接影响班级管理、课程安排、成绩分析和学生发展记录。学校管理人员需要快速了解某个年级或班级的学生情况，教师需要掌握课程和成绩相关数据，管理部门也需要通过统计结果了解教学运行状态。因此，建立一套结构清晰、操作简便、数据集中维护的学生信息管理系统具有现实意义。

从管理效率角度看，系统化的信息平台能够减少重复劳动。学生基本信息一旦录入，后续成绩、考勤和学籍记录即可通过学生 ID 建立联系，避免多处录入同一学生姓名和班级信息。通过 PageHelper 分页和 MyBatis 条件查询，管理人员可以按姓名、班级、课程、日期等条件快速定位记录。对于数据量逐渐增多的学校场景，这种查询方式比人工翻阅表格更加稳定。

从数据管理角度看，系统能够把分散业务统一到关系型数据库中。MySQL 表结构为学生、课程、班级、成绩、考勤等实体提供了明确字段，外键关系也使班级、课程、教师等对象之间的联系更加清楚。虽然本项目的安全控制并未使用完整的 Spring Security 框架，但通过角色和权限基础信息表，已经具备进一步扩展访问控制的基础。

从课程设计训练角度看，本项目有利于理解 Java Web 开发的完整流程。学生可以通过代码阅读掌握 Spring Boot 启动配置、控制器请求映射、服务层接口与实现、Mapper 接口与 XML 映射、Thymeleaf 模板渲染、数据库脚本初始化以及统计报表数据组织等内容。论文对这些内容进行梳理，能够体现课程设计从需求分析到系统实现的完整过程。

1.3 系统达到的目标

本系统的总体目标是实现中学生信息管理相关数据的集中维护和基础统计。系统启动后，管理员通过登录页面进入后台，使用左侧导航访问不同业务模块。每个模块围绕列表展示、条件查询、新增或修改、删除等常见操作展开，使管理人员能够用统一交互方式完成不同类型数据的维护。

在功能目标上，系统需要支持学生信息、年级信息、班级信息、课程信息、成绩

信息、课程安排、教师信息、部门信息、学籍状态和考勤记录等核心数据的管理。学生模块记录姓名、年龄、性别、班级和年级；成绩模块记录学生、课程、成绩、考试类型和日期；考勤模块记录学生、日期、状态和原因；课程安排模块建立班级、课程、教师、星期、节次和教室之间的关系。

在技术目标上，系统应保持清晰的分层结构。控制器层负责处理浏览器请求和页面跳转，服务层封装业务逻辑，Mapper 层通过 MyBatis 执行 SQL，页面层通过 Thymeleaf 展示数据。系统还应具备分页查询能力，避免一次性加载大量记录；在删除存在关联关系的数据时，应尽量处理相关业务表，减少数据不一致问题。

1.4 系统开发工具及环境

本项目运行在 Java Web 开发环境中，后端语言为 Java，工程管理工具为 Maven，框架采用 Spring Boot 2.3.3.RELEASE。数据库使用 MySQL，连接池采用 Druid，持久层使用 MyBatis，分页功能使用 PageHelper。页面端采用 Thymeleaf 模板引擎，静态资源中包含 AdminLTE、Bootstrap、jQuery 和 ECharts 等前端资源，其中统计页面主要以 AdminLTE 信息框和 HTML 表格方式展示汇总数据。

系统配置文件 application.properties 中设置了服务端口、数据库连接、MyBatis 映射文件位置、Thymeleaf 缓存和 SQL 初始化脚本等内容。项目入口类 App 使用 @SpringBootApplication 标注启动类，并通过 @MapperScan 扫描 Mapper 接口，保证服务启动时可以加载控制器、服务类和持久层组件。

1.4.1 Java 简介

Java 是一种面向对象的程序设计语言，具有跨平台、类库丰富和生态成熟等特点。Java 程序通过 JDK 进行编译和运行，依靠 JVM 屏蔽不同操作系统之间的差异。在 Web 系统开发中，Java 常与 Spring 框架、数据库访问框架和模板引擎配合使用，用于构建稳定的后端业务服务。

本系统中的 Java 代码主要分布在 controller、service、mapper 和 domain 等包中。domain 包定义学生、年级、班级、课程、成绩、教师、部门、角色、权限等实体对象和查询对象；controller 包定义页面请求入口；service 包定义业务接口；service.impl 包给出接口实现；mapper 包与 XML 映射文件共同完成数据库访问。这种组织方式体现了 Java Web 项目常见的分层思想。

在具体实现中，Java 对象承担了页面表单数据、数据库记录和业务处理结果之间的传递角色。例如 Student 对象对应学生表主要字段，Score 对象对应成绩记录，Schedule 对象对应课程安排。控制器接收到这些对象后交给服务层处理，服务层再调用 Mapper 方法执行查询或更新，从而完成从页面到数据库的完整数据流转。

1.4.2 MySQL 简介

MySQL 是一种常用的关系型数据库管理系统，适合保存结构化业务数据。它通过数据库、数据表、字段、主键、外键和索引等机制组织数据，并使用 SQL 语言完成数据定义、查询、插入、更新和删除操作。对于中学生信息管理系统而言，学生、课程、班级、成绩、考勤等对象天然具有较明确的结构关系，因此使用关系型数据库较为合适。

本项目的 schema.sql 脚本中定义了学生表、部门表、年级表、班级表、课程表、成绩表、教师表、学籍状态表、角色表、权限表、数据字典表、课程安排表和考勤表。部分表之间设置外键，如班级表关联年级表，成绩表关联学生表和课程表，课程安排表关联班级表、课程表和教师表。通过这些关系可以表达学校管理中较真实的业务联系。

MySQL 与 MyBatis 配合使用时，SQL 语句写在 Mapper XML 文件中，Java 代码通过 Mapper 接口调用这些 SQL。项目中的多个 Mapper XML 使用动态 where 条件实现按名称、编号、状态等字段筛选，并使用 left join 补充关联对象名称。这种方式既保留了 SQL 的直观性，也便于在 Java 代码中调用数据库操作。

在实际运行过程中，数据库连接配置集中保存在 application.properties 中，包含 JDBC 连接地址、用户名、密码、驱动类和连接池类型等信息。系统启动时会根据这些配置创建数据源，后续控制器不直接操作数据库，而是通过服务层和 Mapper 层间接完成数据访问，使数据库连接细节与页面请求处理分离。

本项目使用的 Druid 连接池能够对数据库连接进行统一管理，避免每次请求都重复创建和释放连接。对于课程设计系统而言，连接池不仅可以提高访问效率，也能帮助开发者理解 Web 应用中资源复用的重要性。若后续需要观察 SQL 执行情况，还可以进一步启用连接池监控页面或日志配置。

从数据维护角度看，MySQL 脚本和 Java 对象需要保持一致。如果表字段发生变化，domain 对象、Mapper XML、页面表单和列表字段都可能需要同步调整。本系统表结构较直观，字段命名与业务含义基本对应，因此在论文中可以较清楚地从数据库结构反推系统功能。

在系统实现中，数据库并不只是保存最终结果，还承担了连接不同业务模块的作用。例如学生表中的主键会被成绩表、考勤表和学籍状态表反复引用，课程表中的主键会出现在成绩表和课程安排表中，教师表中的主键会出现在课程安排表中。正是这些主键和外键关系，使系统能够从单条记录扩展到完整业务链条。

MySQL 字段类型的选择也会影响系统维护效果。学生姓名、课程名称、部门名称等信息使用 VARCHAR 保存，成绩和节次等数值使用 INT 保存，创建时间和更新时间使用 TIMESTAMP 保存。不同字段采用合适的数据类型，可以减少无效数据输入，也便于后续排序、筛选和统计。

从课程设计角度看，MySQL 部分的学习价值不仅在于会创建表，还在于理解表与业务之间的对应关系。只有先明确系统需要保存哪些对象、对象之间如何关联，后续 Java 代码中的实体类、Mapper 查询和页面表单才有清晰依据。因此，数据库设计是本系统从需求分析走向功能实现的重要基础。

第 2 章 系统分析设计

2.1 可行性分析

可行性分析主要从技术条件、运行条件和使用对象等方面判断系统是否具备实现基础。本项目的功能范围围绕中学生信息管理中的常见业务，技术上采用成熟的 Java Web 开发框架，数据库结构相对清晰，页面交互以后台管理系统常见的列表、表单和按钮操作为主，因此整体实现难度适中，具备较好的课程设计可行性。

2.1.1 技术可行性

从后端技术看，Spring Boot 能够简化传统 Spring 项目的配置工作，使开发者可以通过较少配置启动 Web 服务并整合 MVC、JDBC、MyBatis 等组件。项目中 App 启动类承担入口作用，控制器通过@Controller 和@RequestMapping 声明请求路径，服务类通过@Service 交由 Spring 容器管理，Mapper 接口通过@MapperScan 进行扫描，技术路线清晰可行。

从数据库访问看，MyBatis 适合本项目这类以管理系统增删改查为主的场景。开发者可以在 XML 文件中直接编写 SQL，并通过动态标签控制查询条件。项目中 StudentMapper、ScoreMapper、ScheduleMapper 等映射文件均采用类似模式，使不同模块可以共享较统一的查询和分页思路。PageHelper 则在列表页面中提供分页支持，降低一次性加载数据的压力。

从页面实现看，Thymeleaf 模板能够直接读取后端传入的模型数据并生成 HTML 页面。项目模板目录中包含 student、grade、class、course、score、schedule、attendance、studentStatus、employee、department、role、permission 和 report 等页面目录，说明系统页面与后端模块保持一一对应关系。前端采用 AdminLTE 风格，使后台管理界面具备较清晰的导航和表格展示能力。

2.1.2 运行可行性

系统运行所需环境较容易满足。开发或演示时只需安装 JDK、Maven 和 MySQL，并保证数据库账号、密码、端口与配置文件一致即可。项目配置中服务端口为 81，数据库名为 rbac，启动后可通过浏览器访问后台页面。由于系统采用 B/S 架构，使用者无需安装专门客户端，只需通过浏览器操作即可。

从使用对象看，本系统面向学校管理人员或课程设计演示人员，页面功能以后台管理系统常见交互为主。用户进入系统后可以通过左侧菜单切换模块，在列表页面中查看数据，通过新增或编辑窗口维护记录，通过删除按钮移除无效数据。只要使用者熟悉基本网页操作，经过简单说明即可完成日常使用。

从维护角度看，系统代码目录结构较规整，模块之间命名较清楚。若后期需要扩展功能，可以按照现有模式增加 domain 对象、Mapper 接口、Mapper XML、Service 接口、Service 实现和 Controller，再补充对应 Thymeleaf 页面即可。数据库方面也可以在现有表结构基础上增加字段或新表，以满足更复杂的学校管理需求。

2.2 功能需求分析

根据源代码结构和页面模板，本系统的功能需求可以划分为登录认证、基础资料维护、学生业务管理、教学业务管理、角色权限基础信息管理和统计报表六类。登录认证负责后台入口控制；基础资料维护负责年级、班级、课程、教师和部门等信息；学生业务管理负责学生档案和学籍状态；教学业务管理负责成绩、考勤和课程安排；角色权限模块负责角色与权限基础数据；统计报表模块负责对关键数据进行汇总展示。

登录模块通过 LoginController 提供根路径、登录、首页和退出功能。当前实现中用

用户名和密码采用固定校验方式，登录成功后把用户标识保存到 Session 并跳转到 index 页面，退出时清除会话信息。该设计适合课程设计演示，但若用于真实生产环境，还需要进一步接入用户表、密码加密、角色授权和访问拦截机制。

基础资料模块包括年级管理、班级管理、课程管理、部门管理和教师管理。年级表保存年级名称和描述，班级表通过 grade_id 关联年级，课程表保存课程名称、课程代码和学分，部门表保存部门名称和编号，教师表通过 department_id 关联部门。这些数据是后续学生、课程安排、成绩和报表统计的基础。

学生业务模块包括学生信息和学籍状态。学生表保存姓名、年龄、性别、班级名称和年级等基本信息；学籍状态表保存学生 ID、状态、原因、操作日期和操作员。通过这两个模块，系统既能维护学生当前基础档案，也能记录入学、转学、毕业等状态变化，从而形成学生过程性管理数据。

教学业务模块包括成绩、考勤和课程安排。成绩表建立学生与课程之间的成绩记录，考勤表记录学生在指定日期的出勤状态，课程安排表建立班级、课程、教师、星期、节次和教室之间的教学安排。这些模块共同支撑学校日常教学运行管理，也为统计报表提供数据来源。

角色权限模块包括角色管理和权限管理。角色表保存角色名称、角色编码和描述，权限表保存权限名称、权限编码、URL、图标、类型和父级权限。根据代码实际情况，该模块目前主要实现角色与权限基础信息的维护，并未形成完整的角色权限关联和访问控制闭环。因此本文在分析时将其定位为 RBAC 扩展基础，而不是完整权限框架。

统计报表模块由 ReportController 实现，包含学生统计、成绩统计、考勤统计、教师统计和课程安排统计。该模块通过调用相关 Service 获取数据后，在 Java 代码中进行分组、计数、平均值等计算，再把统计结果传递给模板页面展示。报表可以帮助管理人员快速了解学生数量、成绩分布、考勤状态、教师部门和课程安排情况。

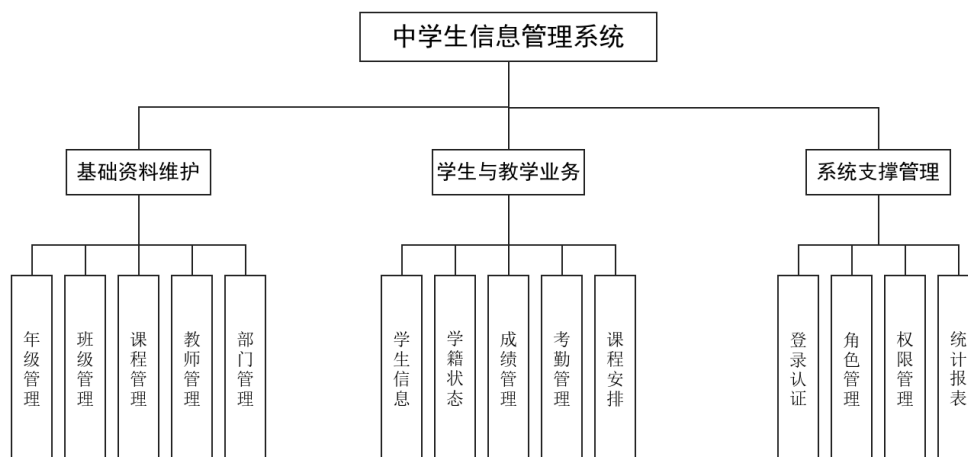


图 2.1 中学生信息管理系统功能模块图

从功能结构图可以看出，系统虽然包含多个业务模块，但各模块并不是孤立存在的。基础资料模块为学生管理、课程安排和教师管理提供基础数据；学生管理模块又为成绩、考勤和学籍状态提供学生来源；统计报表模块则通过读取多个业务模块的数据形成汇总结果。这种结构使系统具有较清晰的数据流向，也便于后期按照模块逐步

扩展。

表 2.1 系统主要控制器路径表（一）

控制器	主要路径	功能说明
LoginController	/	进入登录页
LoginController	/login	提交登录信息
StudentController	/student/list	学生信息列表
GradeController	/grade/list	年级信息列表
ClazzController	/class/list	班级信息列表
CourseController	/course/list	课程信息列表
ScoreController	/score/list	成绩信息列表

表 2.2 系统主要控制器路径表（二）

控制器	主要路径	功能说明
AttendanceController	/attendance/list	考勤记录列表
StudentStatusController	/studentStatus/list	学籍状态列表
ScheduleController	/schedule/list	课程安排列表
EmployeeController	/employee/list	教师信息列表
DepartmentController	/department/list	部门信息列表
RoleController	/role/list	角色基础信息列表
PermissionController	/permission/list	权限基础信息列表

控制器路径表说明系统页面入口与业务模块基本保持一致。除登录控制器和统计报表控制器外，大多数业务控制器都使用模块路径作为一级路径，并在该路径下提供列表、保存更新和删除操作。这样的设计使页面跳转和代码定位更直接，也方便管理人员从系统导航中理解各模块的功能范围。

在需求分析阶段，还需要关注不同模块之间的数据依赖。学生信息是成绩、考勤和学籍状态的基础；班级和年级是学生归属的基础；课程和教师是课程安排的基础；部门是教师分类的基础。只有先保证基础资料准确，后续业务模块才能稳定运行。因此系统菜单虽然按模块划分，但实际使用时存在一定先后顺序。

从用户操作流程看，管理人员通常先维护年级、班级、课程、部门和教师等基础数据，再录入学生信息，之后才能继续维护成绩、考勤、学籍状态和课程安排。统计报表位于业务流程后端，用于对已经录入的数据进行汇总分析。论文在描述功能需求时按照这种数据流向展开，可以使系统逻辑更加清晰。

从异常处理需求看，系统在删除主数据时应考虑关联数据。例如删除学生可能影响成绩、考勤和学籍记录，删除课程可能影响成绩和课程安排，删除班级可能影响课程安排。项目服务层已经对部分关联删除进行了处理，这说明需求设计中不仅要考虑新增和查询，也要考虑数据维护过程中的一致性问题。

第3章 数据库设计

数据库设计是本系统实现的关键。中学生信息管理系统需要长期保存学生、班级、课程、成绩、考勤、教师和权限等数据，因此数据库不仅要能完成基本存储，还要能够表达不同业务对象之间的联系。根据项目 `schema.sql` 脚本，系统使用 MySQL 数据库，数据表采用 InnoDB 引擎和 utf8mb4 字符集，主键多使用 BIGINT 自增字段，时间字段用于记录创建和更新时间。

3.1 数据库概念设计

概念设计阶段主要从业务对象出发抽象系统实体及其关系。系统核心实体包括学生、年级、班级、课程、成绩、教师、部门、学籍状态、考勤记录、课程安排、角色和权限。其中，学生是系统的核心管理对象，成绩、考勤和学籍状态均围绕学生展开；班级依附于年级，课程与教师通过课程安排建立联系；角色和权限则为系统后续权限控制提供基础。

学生与成绩、考勤、学籍状态均为一对多关系，一个学生可对应多条成绩、考勤及学籍状态记录；课程与成绩同样是一对多关系，一门课程可对应多名学生的成绩记录。年级与班级、班级与课程安排、部门与教师之间也均为一对多关系，课程安排同时关联班级、课程和教师，用于描述班级在指定时间、地点的授课安排。

角色与权限在本项目中以基础数据表形式存在，分别保存其基本属性。由于 `schema.sql` 中删除了 `role_permission` 表且未重新建立关联表，说明系统尚未实现角色与权限的多对多绑定及访问控制。因此，在概念设计中仅将角色和权限作为系统安全扩展相关实体进行说明，以保证论文内容与系统实际实现保持一致。

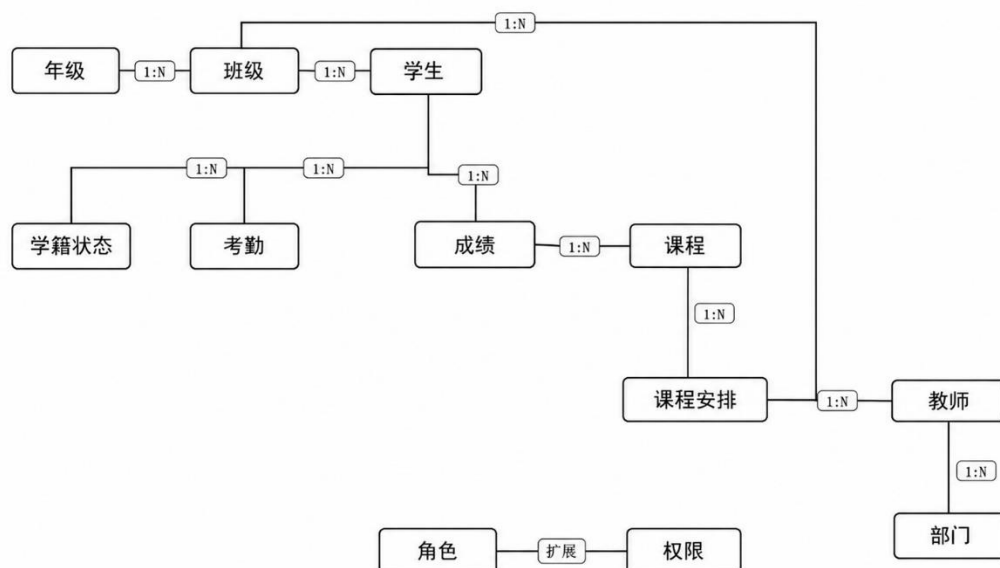


图 3.1 中学生信息管理系统实体关系图

除总体实体关系外，为进一步说明各实体内部属性结构，下面分别绘制学生、年级、班级、课程、成绩、考勤、学籍状态、教师、部门、课程安排、角色和权限等实体属性图。

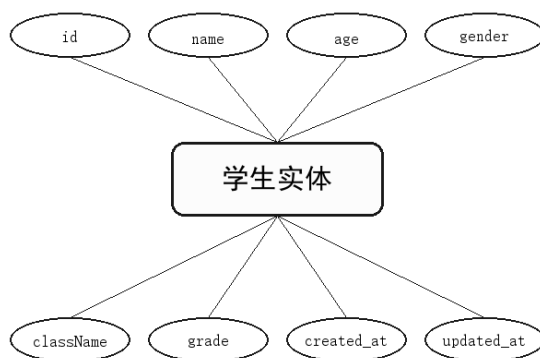


图 3.2 学生实体属性图

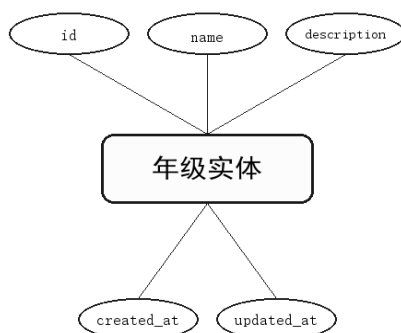


图 3.3 年级实体属性图

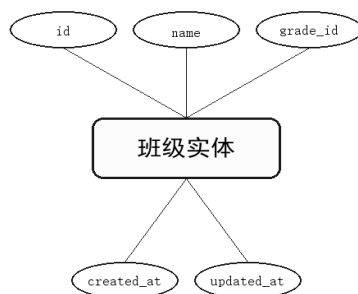


图 3.4 班级实体属性图

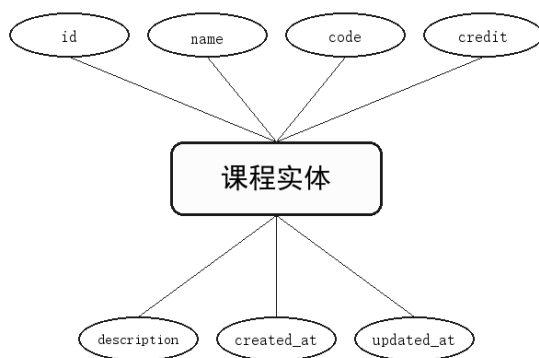


图 3.5 课程实体属性图

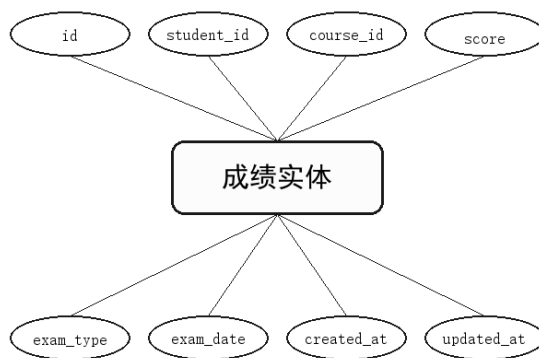


图 3.6 成绩实体属性图

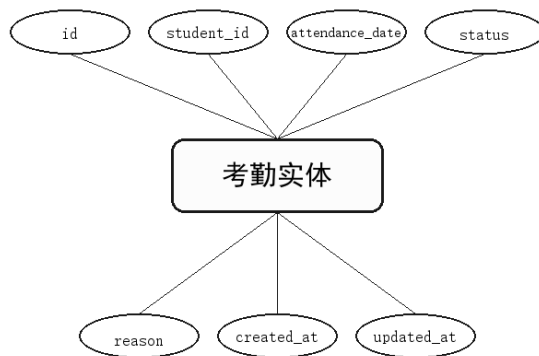


图 3.7 考勤实体属性图

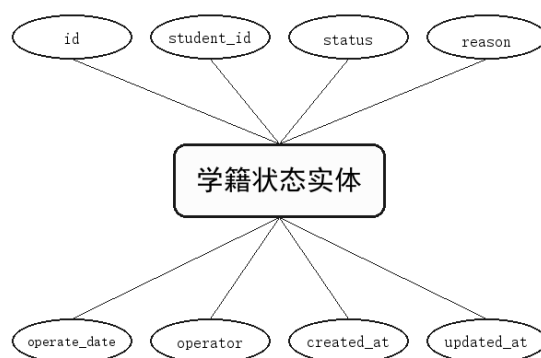


图 3.8 学籍状态实体属性图

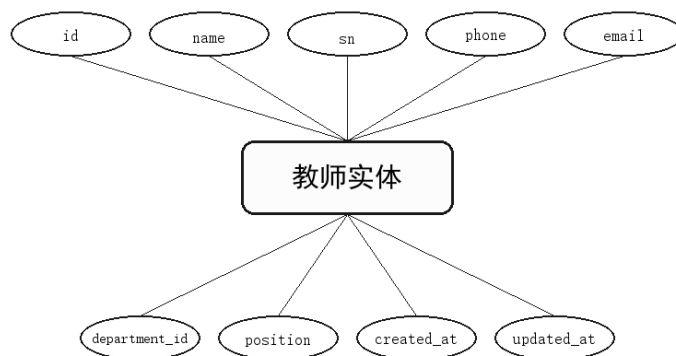


图 3.9 教师实体属性图

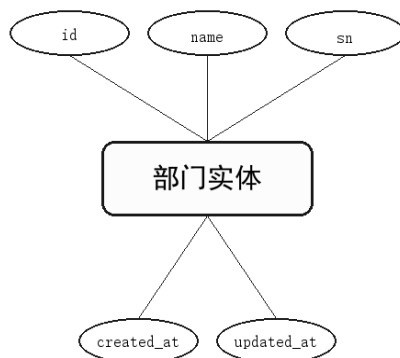


图 3.10 部门实体属性图

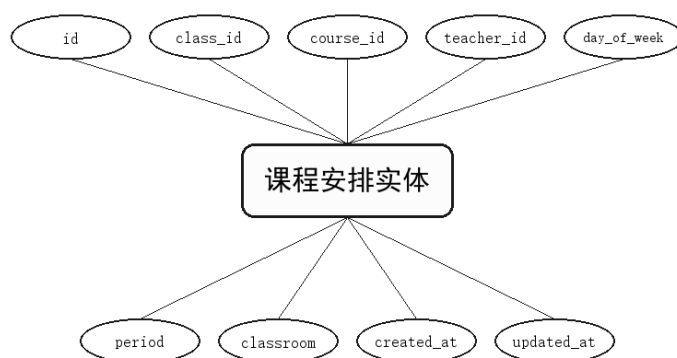


图 3.11 课程安排实体属性图

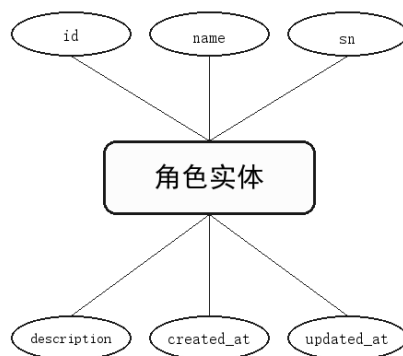


图 3.12 角色实体属性图

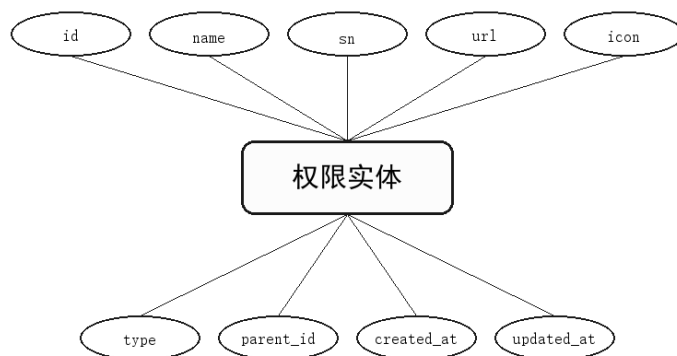


图 3.13 权限实体属性图

3.2 数据库逻辑设计

逻辑设计阶段需要把概念实体转化为具体数据表，并确定字段名称、数据类型和字段含义。本系统的数据表结构与源代码中的 domain 对象和 Mapper XML 文件保持对应关系。表中 created_at 和 updated_at 字段用于记录数据创建和更新时间；存在关联关系的字段如 grade_id、student_id、course_id、department_id、class_id 和 teacher_id 等，用于连接不同业务表。

(1)学生表用于存储 student 相关业务数据，是系统数据库逻辑结构的重要组成部分。

表 3.1 学生表

字段名	数据类型	默认值	描述
id	BIGINT	NULL	主键 ID
name	VARCHAR(50)	NULL	学生姓名
age	INT	NULL	年龄
gender	VARCHAR(10)	NULL	性别
className	VARCHAR(50)	NULL	班级
grade	VARCHAR(20)	NULL	年级
created_at	TIMESTAMP	NULL	创建时间
updated_at	TIMESTAMP	NULL	更新时间

学生表是系统业务的中心表。成绩、考勤和学籍状态均通过 student_id 与学生建立联系，因此学生数据的准确性会直接影响多个业务模块的展示结果。

(2)年级表用于存储 grade 相关业务数据，是系统数据库逻辑结构的重要组成部分。

表 3.2 年级表

字段名	数据类型	默认值	描述
id	BIGINT	NULL	主键 ID
name	VARCHAR(50)	NULL	年级名称
description	VARCHAR(200)	NULL	备注
created_at	TIMESTAMP	NULL	创建时间
updated_at	TIMESTAMP	NULL	更新时间

年级表主要为班级表提供上级分类。虽然字段数量较少，但它可以帮助学校按照初一、初二、初三等维度组织班级和学生数据。

(3)班级表用于存储 clazz 相关业务数据，是系统数据库逻辑结构的重要组成部分。

表 3.3 班级表

字段名	数据类型	默认值	描述
id	BIGINT	NULL	主键 ID
name	VARCHAR(50)	NULL	班级名称
grade_id	BIGINT	NULL	所属年级 ID
created_at	TIMESTAMP	NULL	创建时间
updated_at	TIMESTAMP	NULL	更新时间

班级表通过 grade_id 关联年级，承担年级与学生之间的连接作用。课程安排模块也会通过班级 ID 确定某一节课程面向的班级。

(4)课程表用于存储 course 相关业务数据，是系统数据库逻辑结构的重要组成部分。

表 3.4 课程表

字段名	数据类型	默认值	描述
id	BIGINT	NULL	主键 ID
name	VARCHAR(50)	NULL	课程名称
code	VARCHAR(50)	NULL	课程代码
credit	INT	NULL	学分
description	VARCHAR(200)	NULL	备注
created_at	TIMESTAMP	NULL	创建时间
updated_at	TIMESTAMP	NULL	更新时间

课程表用于保存学校课程基础资料。成绩表和课程安排表都会引用课程数据，因此课程代码、课程名称等字段需要保持规范。

(5)成绩表用于存储 score 相关业务数据，是系统数据库逻辑结构的重要组成部分。

表 3.5 成绩表

字段名	数据类型	默认值	描述
id	BIGINT	NULL	主键 ID
student_id	BIGINT	NULL	学生 ID
course_id	BIGINT	NULL	课程 ID
score	INT	NULL	成绩
exam_type	VARCHAR(50)	NULL	考试类型
exam_date	VARCHAR(20)	NULL	考试日期
created_at	TIMESTAMP	NULL	创建时间
updated_at	TIMESTAMP	NULL	更新时间

成绩表反映学生不同课程和考试类型下的成绩结果。通过关联学生表和课程表，页面可以同时展示学生姓名、课程名称和分数信息。

(6)考勤表用于存储 attendance 相关业务数据，是系统数据库逻辑结构的重要组成部分。

表 3.6 考勤表

字段名	数据类型	默认值	描述
id	BIGINT	NULL	主键 ID
student_id	BIGINT	NULL	学生 ID
attendance_date	VARCHAR(20)	NULL	考勤日期
status	VARCHAR(20)	NULL	考勤状态
reason	VARCHAR(200)	NULL	原因
created_at	TIMESTAMP	NULL	创建时间
updated_at	TIMESTAMP	NULL	更新时间

考勤表用于记录学生某一日期的出勤情况。状态字段可保存出勤、迟到、早退、请假或旷课等结果，原因字段用于补充说明。

(7)学籍状态表用于存储 student_status 相关业务数据，是系统数据库逻辑结构的重要组成部分。

表 3.7 学籍状态表

字段名	数据类型	默认值	描述
id	BIGINT	NULL	主键 ID
student_id	BIGINT	NULL	学生 ID
status	VARCHAR(20)	NULL	状态
reason	VARCHAR(200)	NULL	原因
operate_date	VARCHAR(20)	NULL	操作日期
operator	VARCHAR(50)	NULL	操作员
created_at	TIMESTAMP	NULL	创建时间
updated_at	TIMESTAMP	NULL	更新时间

学籍状态表用于保存学生入学、转学、毕业等状态变化。该表可以作为学生档案的过程性补充，便于后续追溯学生状态变更原因。

(8)教师表用于存储 employee 相关业务数据，是系统数据库逻辑结构的重要组成部分。

表 3.8 教师表

字段名	数据类型	默认值	描述
id	BIGINT	NULL	主键 ID
name	VARCHAR(50)	NULL	教师姓名
sn	VARCHAR(50)	NULL	工号
phone	VARCHAR(20)	NULL	电话
email	VARCHAR(100)	NULL	邮箱
department_id	BIGINT	NULL	部门 ID
position	VARCHAR(50)	NULL	职位
created_at	TIMESTAMP	NULL	创建时间
updated_at	TIMESTAMP	NULL	更新时间

教师表通过 department_id 关联部门，既能保存教师个人基础信息，也能支持按部门统计教师数量和课程安排情况。

(9)部门表用于存储 department 相关业务数据，是系统数据库逻辑结构的重要组成部分。

表 3.9 部门表

字段名	数据类型	默认值	描述
id	BIGINT	NULL	主键 ID
name	VARCHAR(50)	NULL	部门名称
sn	VARCHAR(50)	NULL	部门编号
created_at	TIMESTAMP	NULL	创建时间
updated_at	TIMESTAMP	NULL	更新时间

部门表用于组织教师数据。对于中学场景，可以把语文组、数学组、英语组等教研组抽象为部门，便于教师信息分类维护。

(10)课程安排表用于存储 schedule 相关业务数据，是系统数据库逻辑结构的重要组成部分。

表 3.10 课程安排表

字段名	数据类型	默认值	描述
id	BIGINT	NULL	主键 ID
class_id	BIGINT	NULL	班级 ID
course_id	BIGINT	NULL	课程 ID
teacher_id	BIGINT	NULL	教师 ID
day_of_week	VARCHAR(10)	NULL	星期
period	INT	NULL	节次
classroom	VARCHAR(20)	NULL	教室
created_at	TIMESTAMP	NULL	创建时间
updated_at	TIMESTAMP	NULL	更新时间

课程安排表同时关联班级、课程和教师，是系统中关联关系最集中的业务表。它能够表达某个班级在指定星期、节次和教室的授课安排。

(11)角色表用于存储 role 相关业务数据，是系统数据库逻辑结构的重要组成部分。

表 3.11 角色表

字段名	数据类型	默认值	描述
id	BIGINT	NULL	主键 ID
name	VARCHAR(50)	NULL	角色名称
sn	VARCHAR(50)	NULL	角色编码
description	VARCHAR(200)	NULL	备注
created_at	TIMESTAMP	NULL	创建时间
updated_at	TIMESTAMP	NULL	更新时间

角色表保存角色名称和角色编码，为后续基于角色的权限控制提供基础。当前项目主要完成角色基础数据维护。

(12)权限表用于存储 permission 相关业务数据，是系统数据库逻辑结构的重要组成部分。

表 3.12 权限表

字段名	数据类型	默认值	描述
id	BIGINT	NULL	主键 ID
name	VARCHAR(50)	NULL	权限名称
sn	VARCHAR(50)	NULL	权限编码
url	VARCHAR(200)	NULL	访问地址
icon	VARCHAR(50)	NULL	图标
type	INT	NULL	类型
parent_id	BIGINT	NULL	父级权限 ID
created_at	TIMESTAMP	NULL	创建时间
updated_at	TIMESTAMP	NULL	更新时间

权限表保存菜单或按钮权限的基础属性，包括访问地址、图标、类型和父级权限。后续若增加角色权限关联表，可进一步形成完整菜单授权结构。

从整体逻辑关系看，学生表是成绩表、考勤表和学籍状态表的基础；课程表和学生表共同支撑成绩表；年级表支撑班级表，班级表又支撑课程安排表；教师表通过部

门表进行组织，并参与课程安排。通过这些逻辑关系，系统能够在页面查询中展示更完整的业务信息，例如成绩列表可以显示学生姓名和课程名称，课程安排列表可以显示班级、课程和教师等关联字段。

数据库逻辑设计完成后，系统各业务模块均能够依托统一的数据结构开展业务处理。通过合理设置主键、外键以及字段约束，不仅保证了学生、课程、教师和班级等基础数据之间的关联关系，也提高了数据查询和维护的准确性。在实际业务处理中，各模块均通过对应的数据表完成信息存储，并借助 MyBatis 映射文件实现数据库访问，使系统能够根据用户请求快速完成数据查询、修改和统计等操作。

此外，数据库表结构与系统各层代码保持一致，实体类负责封装业务数据，Mapper 层负责实现数据库访问，Service 层完成业务逻辑处理，Controller 层负责接收用户请求并返回页面数据，各层之间职责明确、分工清晰。这种设计不仅便于系统功能扩展，也降低了后期维护和升级的复杂度。当学校需要增加新的业务模块时，只需在现有数据库结构基础上增加对应的数据表和业务逻辑，即可实现功能扩展，具有较好的可维护性和扩展能力。

综上所述，本系统数据库设计能够较好地满足学生信息管理系统的数据存储需求，为后续学生管理、课程管理、成绩管理、考勤管理以及统计报表等业务功能的实现提供了稳定可靠的数据基础，同时也为下一章系统功能实现提供了完整的数据支撑。

第 4 章 系统实现

4.1 数据库实现

系统数据库通过 schema.sql 脚本进行初始化。脚本先关闭外键检查并删除可能存在的旧表，再按学生、部门、年级、班级、课程、成绩、教师、学籍状态、角色、权限、数据字典、课程安排和考勤等顺序创建数据表。对于存在外键依赖的表，脚本在字段定义后补充 FOREIGN KEY 约束，使数据库能够表达核心业务对象之间的关联。

在应用启动配置中，spring.datasource.schema 和 spring.datasource.data 分别指向 schema.sql 和 data.sql，说明系统可在启动时初始化表结构和样例数据。Druid 连接池负责数据库连接管理，MyBatis 通过 mapper-locations 加载 XML 映射文件，PageHelper 通过配置项完成 MySQL 分页方言适配。数据库实现与后端持久层配置共同保证各模块能够正常访问数据。

表 4.1 数据库表实现表

数据表	实现内容	关联说明
student	学生基础数据	被成绩、考勤、学籍状态引用
grade/clazz	年级班级数据	班级通过 grade_id 关联年级
course/score	课程与成绩	成绩同时关联学生和课程
employee/department	教师与部门	教师通过 department_id 关联部门
schedule	课程安排	关联班级、课程和教师
role/permission	角色权限基础信息	为后续访问控制扩展提供基础
attendance/student_status	考勤与学籍状态	均以学生为核心关联对象

4.2 功能实现

系统功能实现遵循较统一的模块模式。多数业务控制器都提供 /list、/saveOrUpdate 和 /delete 三个核心请求入口。list 负责接收查询对象、调用服务层分页查询并返回列表页面；saveOrUpdate 根据对象 ID 是否为空决定新增或修改；delete 根据 ID 删除记录并重定向回列表页面。这种模式减少了不同模块之间的实现差异，也便于论文按模块进行说明。

服务层在系统中承担业务调度作用。除简单调用 Mapper 完成保存、更新和查询外，部分服务实现还处理了关联数据删除。例如删除学生时，需要先删除与该学生相关的成绩、考勤和学籍状态记录；删除课程时，需要处理成绩和课程安排；删除年级时，需要处理课程安排和班级；删除部门时，需要处理教师和课程安排。该处理方式体现了系统对业务关联关系的考虑。

页面层采用 Thymeleaf 模板。模板页面通过后端 Model 接收分页结果、查询条件和下拉列表数据，再在 HTML 表格中展示。系统中的学生、成绩、考勤、课程安排等页面均围绕表格列表和编辑表单展开，操作按钮通过链接或表单提交触发控制器请求。由于当前环境未实际裁切运行截图，本节按要求保留空白截图占位，后续可替换为真实页面截图。

表 4.2 系统代码层次说明表（一）

代码层次	主要职责	实现说明
domain	实体对象与查询对象	保存页面表单数据和数据库记录字段
controller	请求入口	接收浏览器请求并返回模板页面
service	业务接口	定义模块可调用的业务方法
service.impl	业务实现	完成分页查询、保存更新和关联删除
mapper	持久层接口	声明数据库访问方法

表 4.3 系统代码层次说明表（二）

代码层次	主要职责	实现说明
mapper XML	SQL 映射	通过动态 where 和 left join 实现查询
templates	页面模板	使用 Thymeleaf 展示列表和表单
static	静态资源	保存样式、脚本、图片和前端组件
schema.sql	表结构脚本	初始化数据库表和外键关系
data.sql	数据脚本	提供系统演示所需的基础样例数据

从代码层次可以看出，系统没有把 SQL、业务判断和页面展示混写在同一文件中，而是按照 Java Web 项目常用方式拆分到不同目录。控制器只负责组织请求和模型数据，服务层负责业务处理，Mapper 层负责 SQL 执行，模板页面负责展示。这种结构虽然增加了一定文件数量，但更有利于后期定位问题和扩展模块。

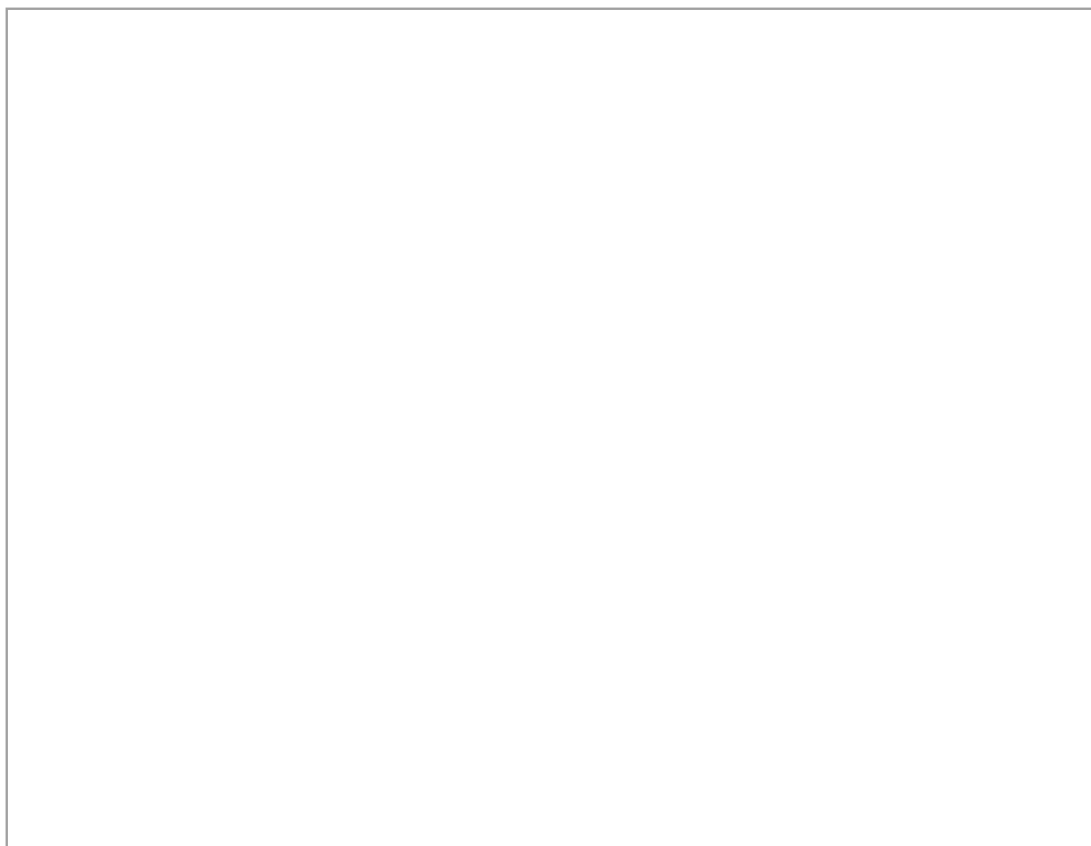


图 4.1 系统登录页面

登录页面对应 `LoginController` 中的 `/login` 请求。系统根据输入的账号和密码判断是否允许进入后台，并把登录成功的用户标识写入 `Session`。虽然当前校验方式较简单，但已经体现出后台系统入口控制的基本流程。

该功能测试时应重点检查错误账号是否被拦截、正确账号是否能够写入会话以及退出后是否回到登录入口。由于当前项目未接入用户表，后续完善时可以把固定校验改为数据库校验，并补充密码加密存储。

图 4.2 对应的功能在系统中通过相应控制器和模板页面实现。该处预留截图位置，后续可在系统运行后按页面边缘裁切并替换为空白占位图。

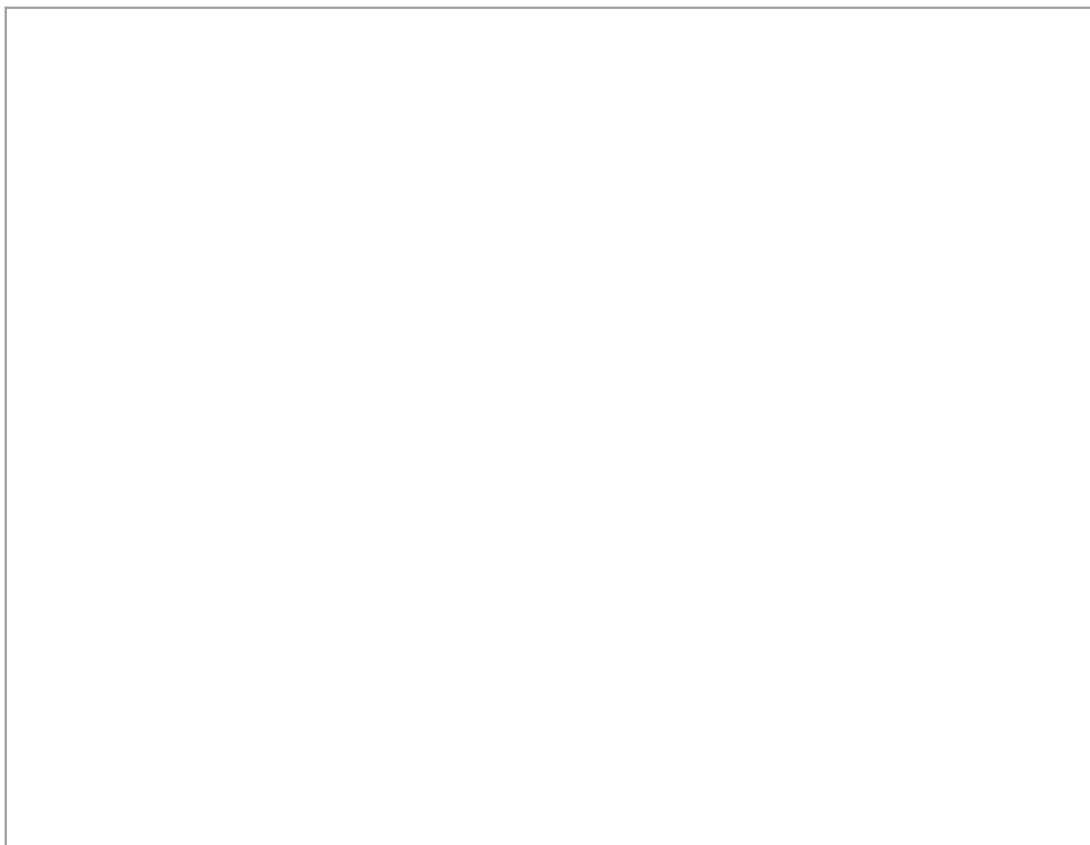


图 4.2 系统后台首页

后台首页用于承载系统导航和各业务模块入口。用户登录成功后进入 `index` 页面，通过菜单访问学生、课程、成绩、考勤、权限和报表等模块，页面结构体现了后台管理系统的常见布局。

后台首页测试时应重点检查菜单链接是否能跳转到对应模块，静态样式和脚本是否加载完整。若某些资源路径错误，页面布局会出现错位，因此首页也是验证前端资源配置是否正确的重要页面。

后台首页作为系统的统一入口，不仅承担页面导航功能，还负责组织各业务模块之间的访问关系。系统采用后台管理系统常见的布局方式，将菜单导航、内容展示以及用户操作区域进行合理划分，使管理人员能够快速定位所需功能。各业务模块按照学生管理、教学管理、基础资料管理和系统管理等类别进行组织，不仅提高了页面的可读性，也增强了系统整体的操作体验。

为了保证后台首页能够稳定运行，系统在页面加载过程中需要同时完成模板渲染、静态资源引用以及菜单数据加载等工作。当用户登录成功后，浏览器首先加载公共样式文件和脚本文件，再根据控制器返回的数据生成首页内容。如果其中任一环节出现异常，都可能导致页面显示不完整或

导航菜单无法正常使用。因此，在系统测试阶段，需要重点验证页面加载速度、菜单响应情况以及各模块之间的跳转是否正常，确保用户能够顺利进入各项业务功能。

此外，后台首页还能够反映系统整体运行状态。当数据库连接正常、控制器映射正确、前端资源完整加载时，各业务模块均可通过导航菜单正常访问。管理人员可以从首页快速进入学生信息、课程信息、成绩管理、考勤管理以及统计报表等页面，实现系统各项业务功能之间的无缝切换，为后续数据维护和业务处理提供统一的操作入口。

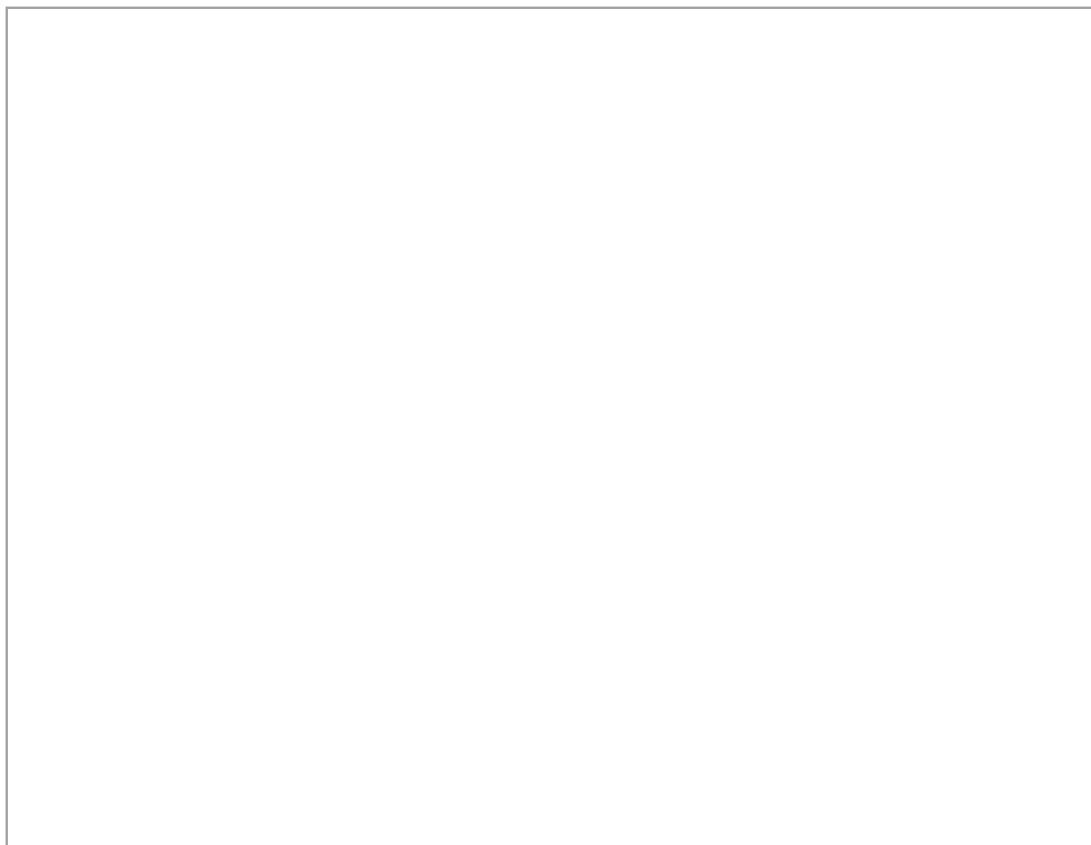


图 4.3 学生信息管理页面

学生信息管理页面对应 `StudentController` 和 `StudentServiceImpl`。列表页面展示学生姓名、年龄、性别、班级和年级等字段，新增或修改时将表单数据封装为 `Student` 对象，再由服务层调用 `Mapper` 写入数据库。

学生模块测试时应分别验证新增、修改、删除和按条件查询。删除学生时还需要观察关联成绩、考勤和学籍状态是否被正确处理，避免数据库中留下无法解释的孤立记录。

图 4.4 对应的功能在系统中通过相应控制器和模板页面实现。该处预留截图位置，后续可在系统运行后按页面边缘裁切并替换为空白占位图。

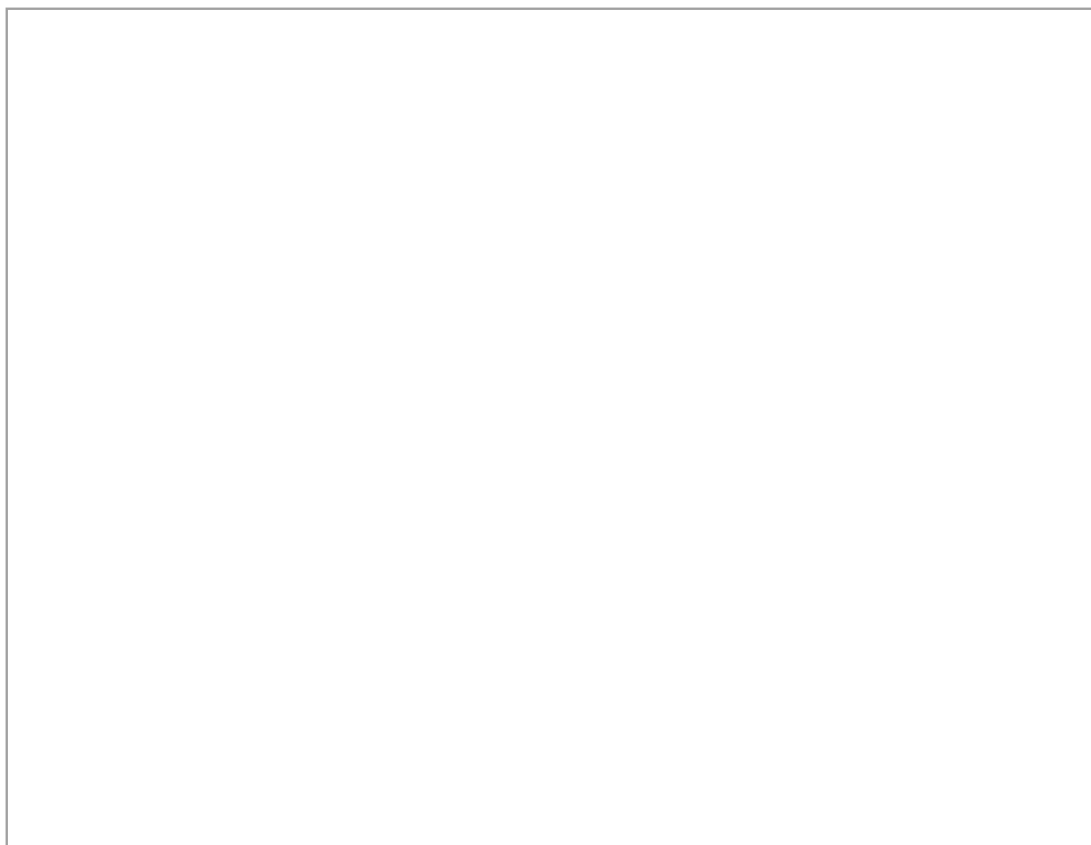


图 4.4 年级与班级管理页面

年级与班级管理页面体现基础资料之间的层级关系。年级表保存年级名称，班级表通过 `grade_id` 关联年级。页面维护这些数据后，学生管理和课程安排模块可以继续使用班级与年级信息。

年级和班级模块测试时应先保存年级，再新增班级并选择所属年级。若班级列表能够显示年级名称，说明 **Mapper** 关联查询和页面数据显示均已正常工作。

学生信息管理模块是整个中学生信息管理系统中使用频率最高的核心业务模块，也是成绩管理、考勤管理、学籍状态管理等功能的数据来源。系统通过统一的学生信息维护机制，实现学生基础档案的集中管理，保证各业务模块引用的学生数据保持一致。当管理员新增学生信息后，系统将自动保存学生姓名、年龄、性别、所属班级及年级等基础资料，为后续成绩录入、考勤登记和课程安排等业务提供可靠的数据支撑。

在页面实现过程中，学生信息列表采用表格形式展示数据，并结合分页查询提高数据浏览效率。当学生数量不断增加时，分页机制能够减少一次性加载的数据量，降低服务器压力，同时提升页面响应速度。管理员还可以结合姓名、班级等条件进行查询，快速定位目标学生，提高日常信息维护效率。

从系统实现角度来看，学生管理模块充分体现了系统分层设计思想。控制层负责接收用户请求并完成页面跳转，业务层负责数据校验和业务处理，持久层负责数据库访问，各层之间职责划分明确。当学生信息发生新增、修改或删除操作时，系统能够及时同步数据库中的相关数据，为其他业务模块提供准确、完整的数据基础，从而保证整个系统运行过程中的数据一致性和可靠性。

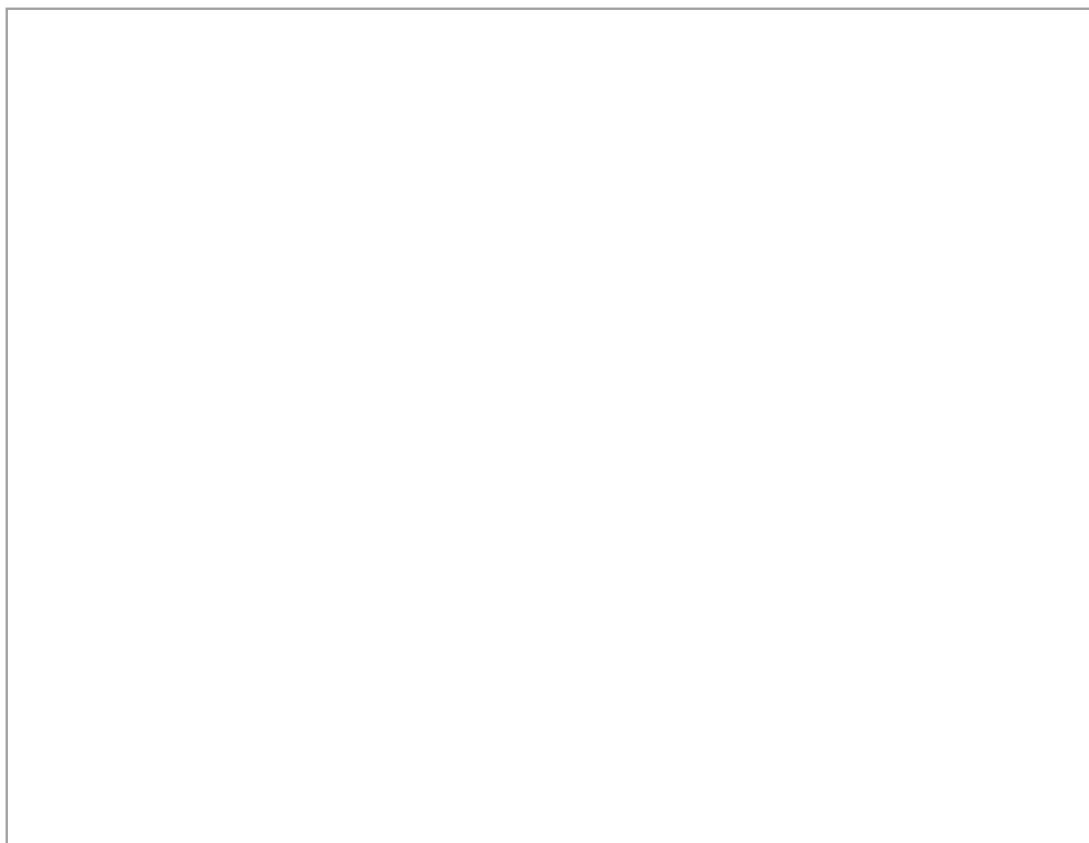


图 4.5 课程与课程安排管理页面

课程与课程安排管理页面分别对应 `CourseController` 和 `ScheduleController`。课程模块维护课程名称、课程代码和学分，课程安排模块进一步把班级、课程、教师、星期、节次和教室组合为教学计划。

课程与课程安排模块测试时应关注外键数据是否完整。课程安排需要同时引用班级、课程和教师，如果任何基础数据缺失，页面下拉选项或保存逻辑都会受到影响。

课程管理与课程安排模块共同构成了系统教学管理的重要组成部分。其中，课程管理负责维护课程基础资料，而课程安排则在课程资料的基础上建立班级、教师、授课时间以及教室之间的对应关系，实现教学资源的统一组织。系统通过各业务数据之间的关联关系，保证课程安排能够准确引用已有课程和教师信息，避免重复录入基础数据，提高系统整体的数据利用效率。

在课程安排页面中，系统采用下拉列表选择课程、班级和教师等基础信息，管理员只需选择已经维护好的数据即可完成课程安排，大幅降低了人工输入带来的错误风险。同时，系统能够按照星期、节次等信息完成课程计划配置，使每个班级在不同时间段均具有明确的授课安排，便于学校开展日常教学管理工作。

从系统测试角度来看，除验证课程信息能够正常新增、修改和删除外，还应重点检查课程安排数据是否能够正确保存，并验证课程、教师和班级之间的关联关系是否保持一致。当修改课程名称或教师信息后，课程安排页面应能够同步显示最新数据，确保各业务模块之间的数据始终保持一致。此外，还应检查不同班级之间课程安排是否存在冲突，为后续教学计划的合理制定提供可靠保障。

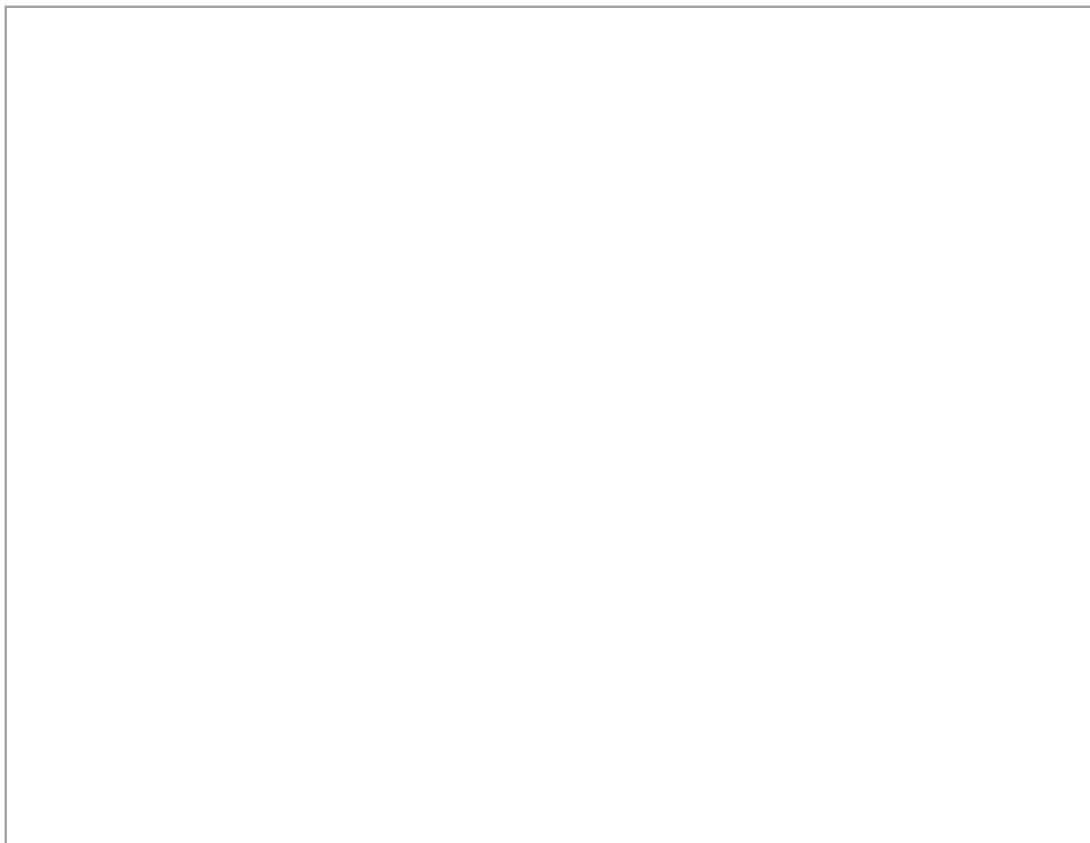


图 4.6 成绩管理页面

成绩管理页面对应 `ScoreController`。新增成绩时需要选择学生和课程并填写考试类型、考试日期和分数。`Mapper` 查询时可通过关联查询展示学生姓名和课程名称，提高列表页面可读性。

成绩模块测试时应关注分数字段的有效性和列表展示的关联信息。较完整的测试应包括正常分数、边界分数和缺失学生或课程时的处理情况，后续可增加前端校验提升可靠性。

成绩管理模块不仅负责学生成绩的录入和维护，还承担着学生学习情况统计与分析的重要任务。系统通过学生信息和课程信息建立成绩关联关系，使每一条成绩记录都能够准确对应到具体学生和课程，避免因重复录入基础信息而造成数据不一致的问题。当管理员完成成绩录入后，系统能够立即更新数据库中的成绩数据，并在列表页面实时展示查询结果，提高成绩管理工作的及时性和准确性。

为了提升数据查询效率，系统在成绩列表展示过程中采用关联查询方式，同时获取学生姓名、课程名称以及考试类型等信息，使管理人员无需分别查询多个业务模块即可掌握完整成绩情况。结合分页查询功能，即使成绩记录数量不断增加，系统仍能够保持较好的页面响应速度和数据展示效果，满足学校日常成绩管理需求。

在系统测试过程中，还需要验证成绩数据修改后的同步更新情况，确保页面展示内容与数据库保存结果保持一致。同时，应检查学生信息、课程信息发生调整后，成绩记录是否仍能够正确关联对应对象，避免出现关联失效或数据显示异常等问题。通过对新增、修改、删除以及查询等功能进行全面测试，可以进一步保证成绩管理模块运行稳定，为成绩统计分析和后续报表生成提供可靠的数据基础。

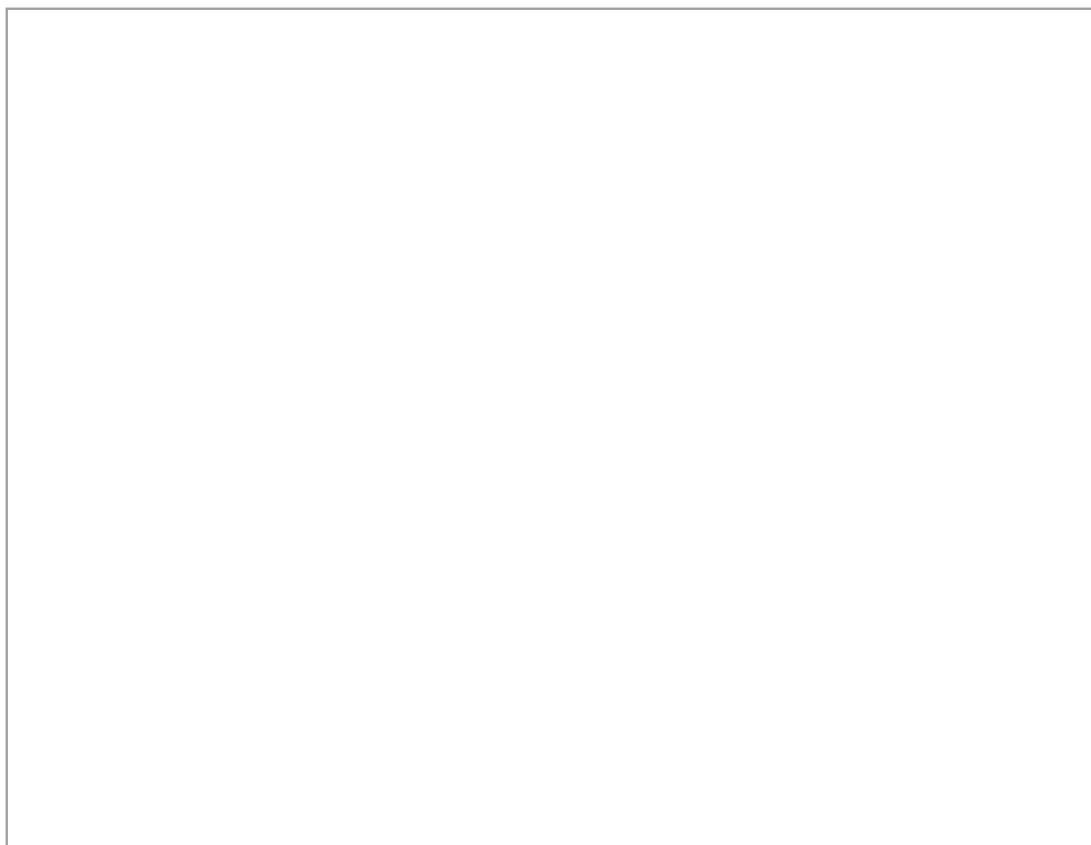


图 4.7 考勤管理页面

考勤管理页面用于保存学生每天的出勤状态。系统支持出勤、迟到、早退、请假、旷课等状态表达，原因字段可以补充请假或异常情况说明，为后续班级管理和统计分析提供依据。

考勤模块测试时应覆盖多种状态值，并检查同一学生不同日期的记录是否可以独立保存。若需要进一步完善，可增加同一学生同一天不能重复录入的业务约束。

考勤管理模块主要用于记录学生日常出勤情况，是学校开展学生日常管理的重要组成部分。系统支持记录考勤日期、考勤状态以及请假原因等信息，并能够根据学生编号建立对应关系，实现学生考勤数据的统一维护。管理员可以通过列表页面快速查询指定日期或指定学生的考勤记录，提高考勤信息管理效率。

为了保证考勤数据准确性，系统在保存数据前需要验证学生信息是否存在，并检查考勤状态是否符合业务要求。测试过程中应重点验证正常出勤、迟到、请假以及旷课等不同状态的录入情况，同时检查重复录入、修改以及删除操作是否能够正确执行，为学校后续考勤统计提供可靠的数据来源。

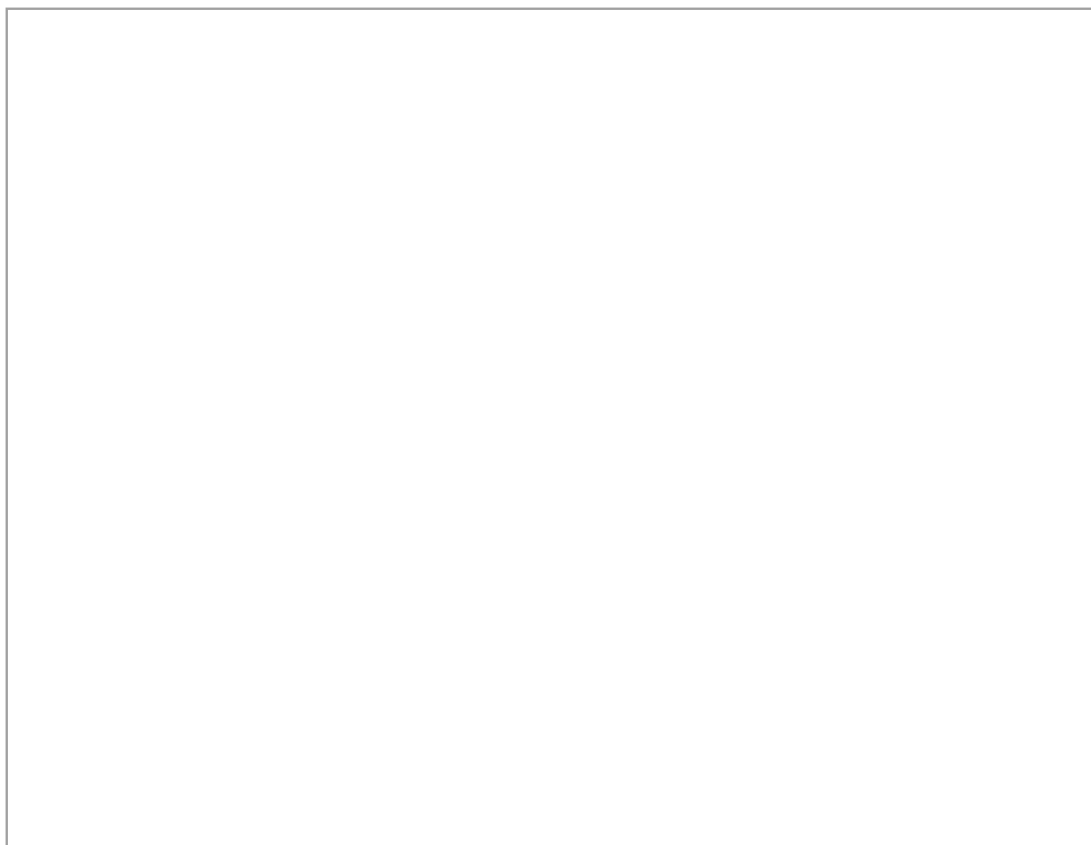


图 4.8 学籍状态管理页面

学籍状态管理页面用于记录学生入学、转学、毕业等状态变化。与只保存当前学生信息相比，学籍状态表可以保存过程性记录，使学校管理人员能够追踪学生状态变更时间和操作人员。

学籍状态模块测试时应关注状态变化的时间顺序和操作员字段。后续扩展时可以把操作员与真实登录用户关联，使学籍变更记录具有更强的可追溯性。

学籍状态管理模块主要记录学生在校期间的重要学籍变化信息，包括入学、转学、休学、复学以及毕业等业务状态。系统通过学生编号建立关联关系，使每条学籍记录均能够对应具体学生，实现学生全过程管理，为学校开展学籍查询提供完整的数据支撑。

系统在实现过程中采用统一的数据维护流程，管理员录入学籍状态后即可保存至数据库，并能够在列表页面实时查看历史记录。测试过程中应重点验证学生信息与学籍状态之间的关联关系是否正确，同时检查不同状态切换后页面是否能够及时更新，确保学籍管理数据真实、完整且具有可追溯性。

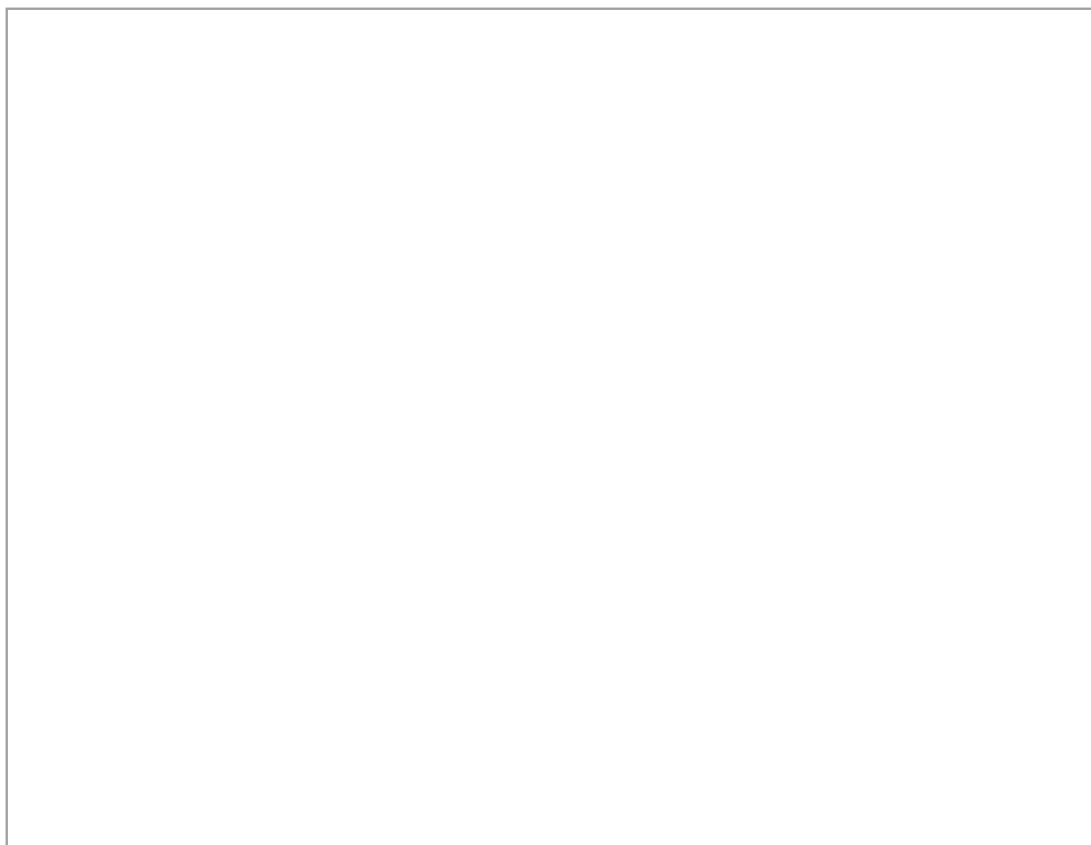


图 4.9 角色权限基础信息管理页面

角色权限基础信息管理页面包含角色列表和权限列表。角色表保存角色名称、编码和说明，权限表保存权限名称、访问地址、图标、类型和父级权限。该模块为后续完善授权提供数据基础。

角色与权限管理模块主要负责系统基础权限数据维护，为后续实现完整的权限控制提供基础支持。角色信息用于描述不同类型用户的身份，权限信息用于定义菜单访问和功能操作范围，两者共同构成系统权限管理的数据基础。

虽然本系统目前主要完成角色和权限基础数据维护，但整体数据结构已经具备进一步扩展权限控制功能的条件。测试过程中主要验证角色、权限信息的新增、修改和删除功能是否正常，同时检查菜单地址、权限编码等关键字段是否能够正确保存，为系统后续权限扩展提供可靠的数据基础。

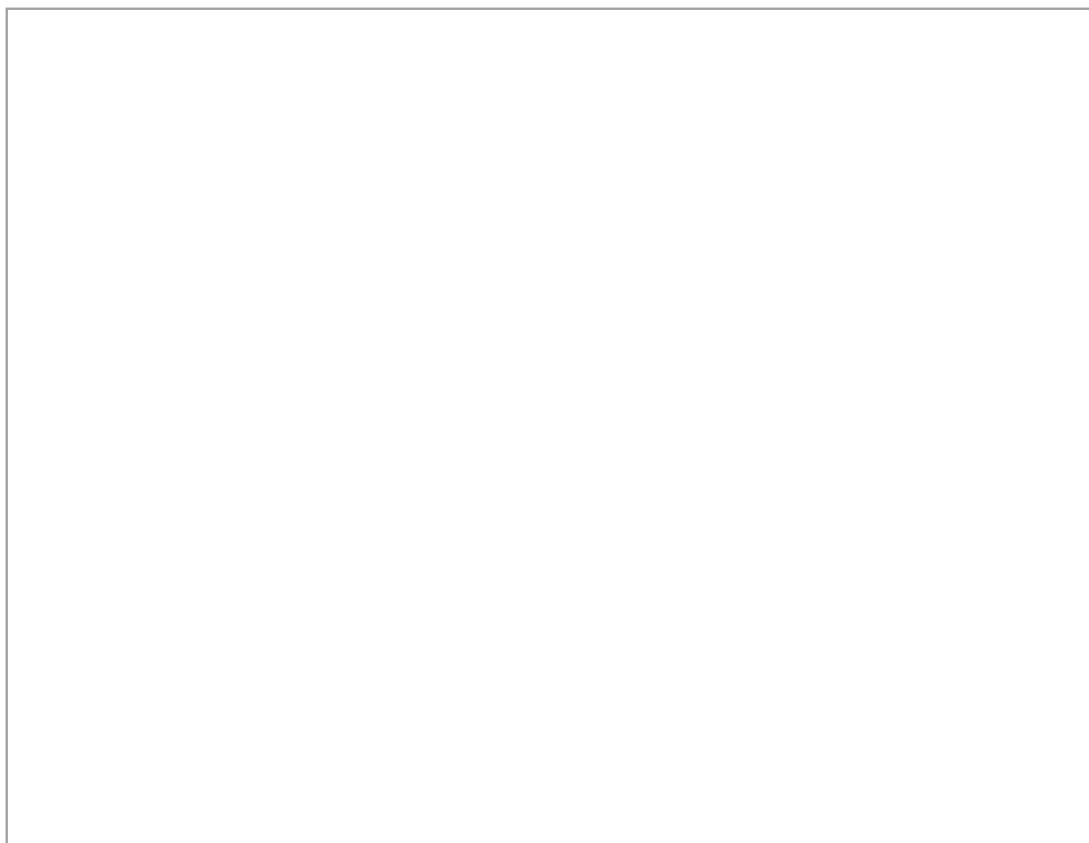


图 4.10 统计报表页面

统计报表页面通过 **ReportController** 实现。控制器从多个 **Service** 中读取业务数据，再使用 **Java** 集合进行分组和汇总，最终将统计结果传递给 **Thymeleaf** 模板展示。

统计报表测试时应先准备足够样例数据，再比较报表指标和原始列表是否一致。若某类数据为空，页面也应能正常展示默认值或空结果，而不应出现异常报错。

统计报表模块主要负责对学生、成绩、教师、课程以及考勤等业务数据进行汇总分析，为管理人员提供直观的数据展示结果。系统通过调用多个业务模块的数据，在服务层完成统计计算，再将结果传递至页面进行展示，使管理人员能够快速了解学校当前教学管理情况。

在系统测试过程中，应重点验证统计结果与数据库原始数据是否保持一致，同时检查新增、修改或删除业务数据后统计页面是否能够及时更新。通过统计报表功能，系统能够进一步提升数据利用价值，为学校教学管理和业务决策提供有效的数据支持。

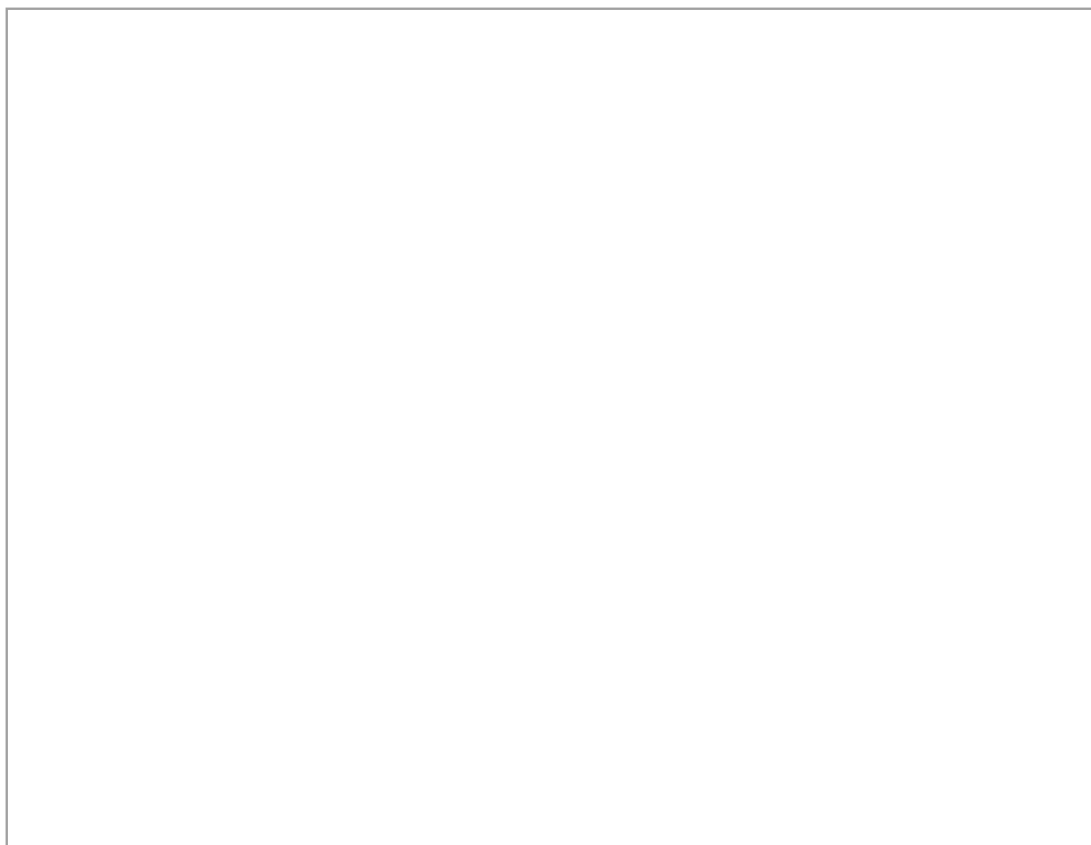


图 4.11 教师信息管理页面

教师信息管理页面对应 **EmployeeController**。教师表保存姓名、工号、电话、邮箱、职位和部门 ID，可与部门表联动展示教师所属部门，并参与课程安排中的授课教师选择。

教师模块测试时应关注教师与部门之间的关联展示。新增教师时选择部门，列表中显示部门名称，可以验证 **department_id** 字段和关联查询是否正确。

教师管理模块承担学校教师基础信息维护任务，是课程安排模块的重要数据来源。系统支持教师姓名、工号、联系电话、所属部门以及岗位等信息的统一管理，通过部门信息建立教师分类关系，方便学校按照不同教研组开展日常管理工作。页面采用统一的列表管理方式，支持新增、修改、删除以及条件查询等常用操作，提高教师信息维护效率。

在系统实现过程中，教师信息通过业务层完成数据校验，再由 **Mapper** 层写入数据库。当课程安排模块引用教师信息时，系统能够自动读取最新教师数据，保证课程安排与教师基础信息保持一致。测试过程中应重点验证教师信息修改后课程安排页面是否能够同步更新，同时检查删除教师时是否存在课程安排引用，避免产生关联数据异常，保证系统整体数据的一致性和完整性。

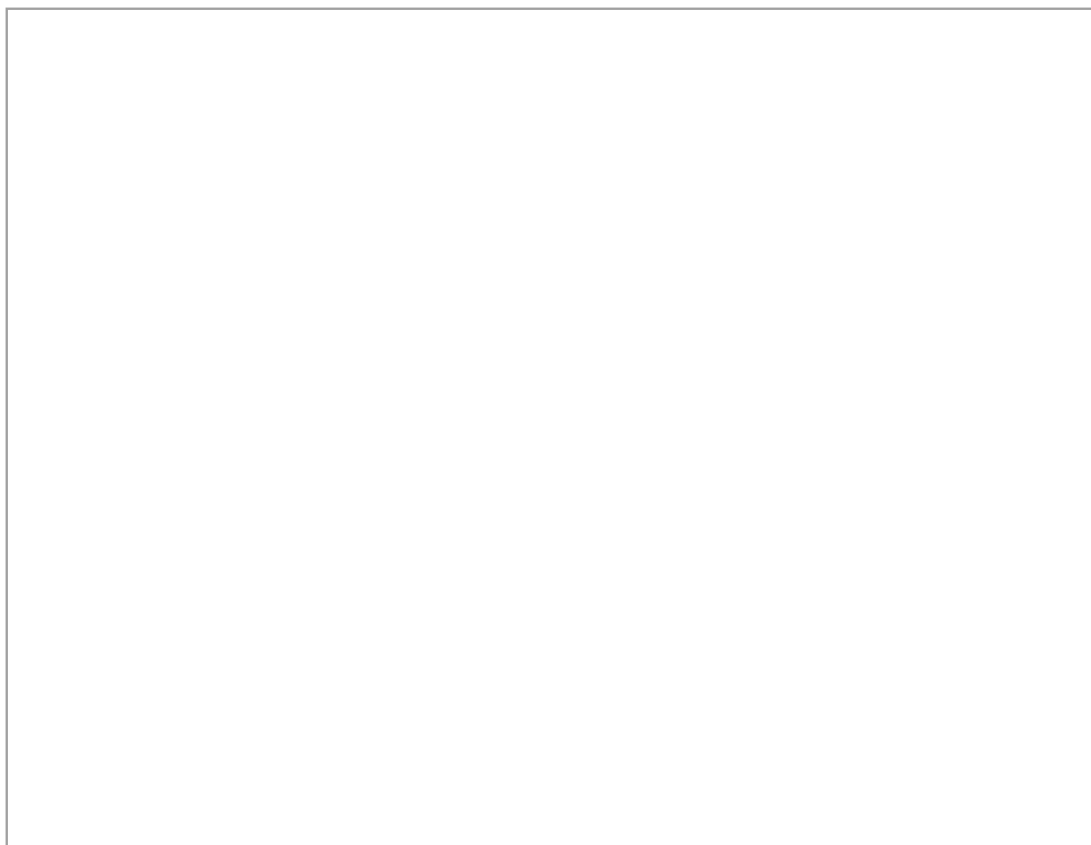


图 4.12 部门信息管理页面

部门信息管理页面对应 `DepartmentController`。部门数据可以表示学校中的教务处、年级组或学科教研组，教师信息通过 `department_id` 与部门关联，从而支持按部门查看教师数量。

部门模块测试时应先维护部门，再维护教师。删除部门前需要考虑教师和课程安排是否仍然引用该部门或部门下教师，避免管理数据之间出现冲突。

部门管理模块主要负责维护学校组织机构信息，为教师管理提供分类依据。系统通过部门编号和部门名称建立基础数据，在教师信息维护过程中能够快速选择所属部门，实现教师与部门之间的关联管理。合理的部门划分不仅有利于教师信息维护，也为后续统计分析提供数据支持。

在系统测试过程中，应重点验证部门新增、修改及删除功能是否正常，同时检查教师信息与部门之间的关联关系是否正确维护。当部门名称发生调整时，教师列表能够同步显示最新部门信息；若删除仍存在教师引用的部门，应及时进行业务处理或数据校验，保证系统运行稳定可靠。

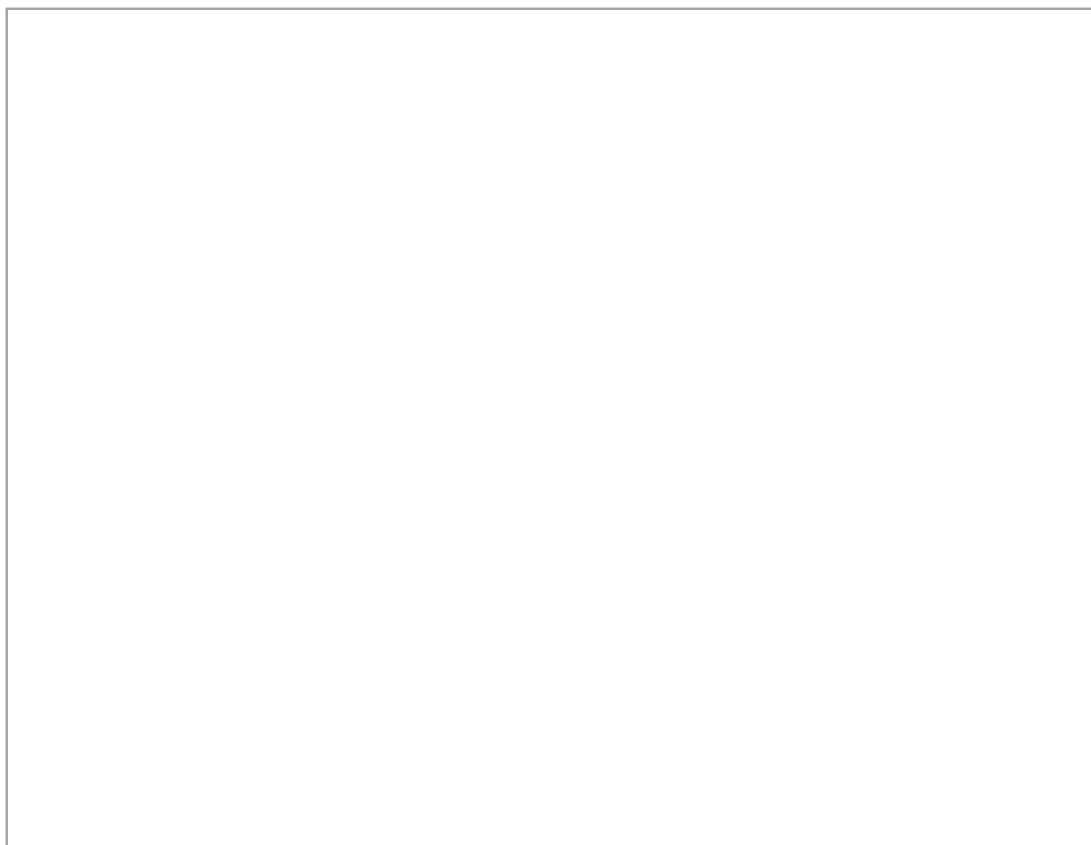


图 4.13 学生统计报表页面

学生统计报表页面主要围绕学生表展开，可按年级、班级和性别等维度进行汇总。该报表有助于管理人员快速掌握学生分布情况，也能辅助班级规模 and 教学资源配置判断。

学生统计报表测试时应检查不同班级、年级和性别维度的统计口径。统计结果不应只显示总数，还应能够辅助管理人员理解学生分布结构。

学生统计报表模块主要用于汇总系统中的学生基础数据，并以统计结果的形式展示各年级、各班级学生分布情况。系统通过读取学生信息表中的相关数据，结合班级和年级基础资料完成统计计算，使管理人员能够快速掌握学校学生数量及分布情况。相比逐条查询学生信息，统计报表能够更加直观地反映整体数据，为学校开展招生管理、班级调整以及教学资源配置提供数据参考。

在系统实现过程中，统计模块主要通过业务层调用学生相关数据，并完成数量统计、分类汇总等处理，再将统计结果返回页面进行展示。测试过程中应重点验证统计结果是否与数据库实际数据保持一致，同时检查新增、删除或修改学生信息后统计数据是否能够及时更新，确保报表内容真实、准确，为系统数据分析提供可靠支持。

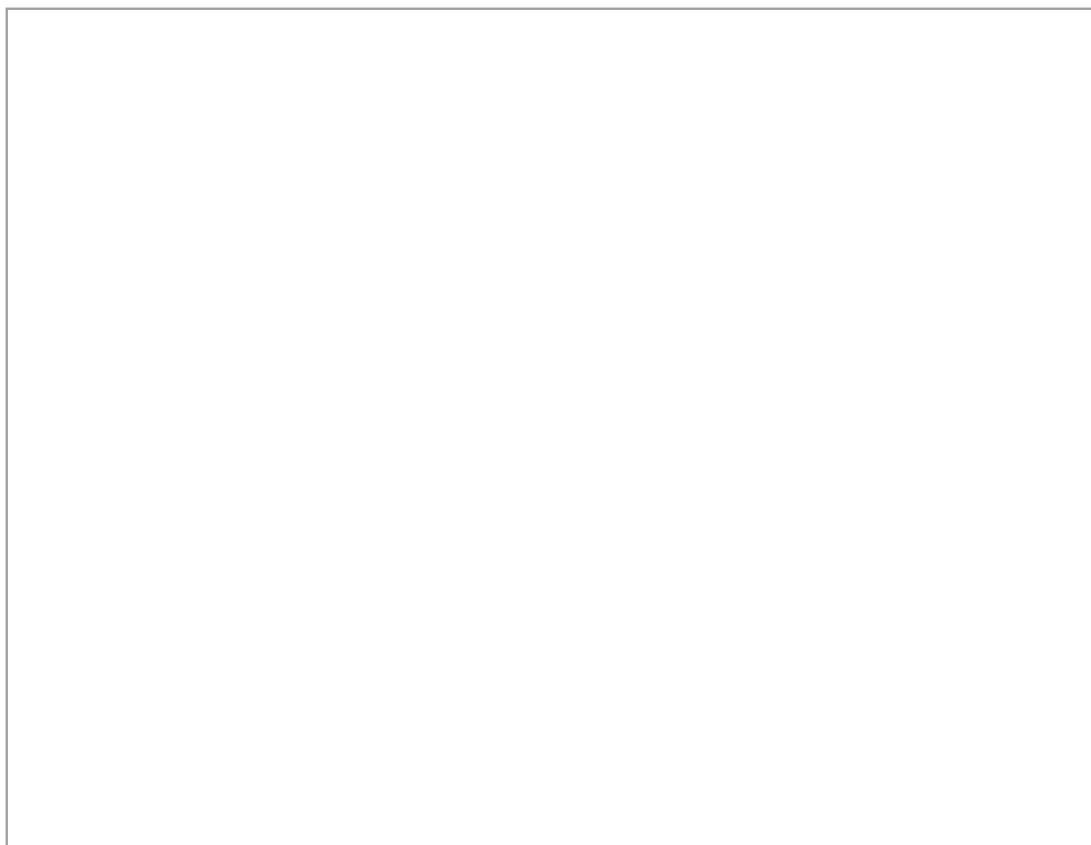


图 4.14 成绩统计报表页面

成绩统计报表页面主要围绕成绩表展开，可计算平均分、最高分、最低分和分数段分布。结合课程表和学生表后，报表可以从课程、学生和考试类型等角度解释成绩数据。

成绩统计报表测试时应关注平均值和分数段统计是否合理。若存在空成绩或异常分数，报表算法需要明确是否排除，以保证统计结果可解释。

成绩统计报表模块主要负责对学生成绩数据进行汇总分析，通过统计不同课程成绩、学生成绩以及平均成绩等信息，帮助管理人员全面了解学生学习情况。系统依据成绩表中的数据完成统计计算，并结合课程信息展示相关结果，使成绩数据不仅能够用于日常维护，还能够发挥数据分析和教学评价作用，提高系统的数据利用价值。

在系统测试过程中，应重点验证统计数据计算是否正确，并检查成绩录入、修改或删除后统计结果是否能够同步更新。同时，还需要验证不同课程成绩之间的统计是否准确，确保页面展示结果与数据库中的实际数据保持一致。通过统计报表功能，系统能够为教师分析教学效果、学校掌握整体学习情况提供更加直观的数据支持，也进一步体现了学生信息管理系统的信息化管理优势。

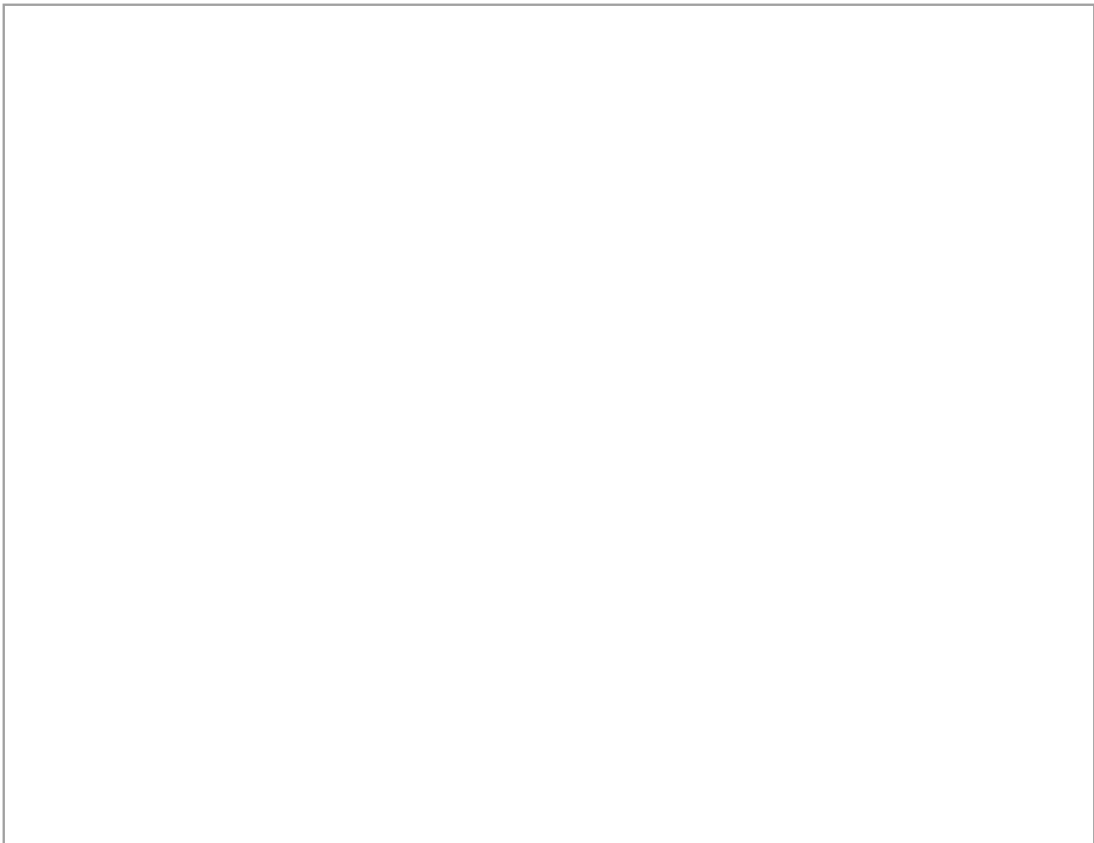


图 4.15 课程安排统计页面

课程安排统计页面围绕 `schedule` 表展开，可统计教师授课安排、班级课程数量和教室使用情况。该页面能帮助管理人员检查课程安排是否完整，并发现同一时间段潜在的资源冲突。

课程安排统计测试时应关注班级、教师和教室三个维度。后续若扩展冲突检测，可根据星期和节次判断同一教师或教室是否被重复安排。

表 4.4 系统功能测试表（一）

测试模块	测试内容	预期结果
登录	输入 <code>admin</code> 和正确密码	进入后台首页
学生查询	按学生姓名或班级筛选	列表显示符合条件的数据
学生新增	填写姓名、年龄、性别等信息	保存后列表出现新记录
课程维护	新增课程名称、代码和学分	课程表成功写入记录
成绩录入	选择学生和课程并填写成绩	成绩列表显示学生与课程名称

表 4.5 系统功能测试表（二）

测试模块	测试内容	预期结果
考勤维护	填写学生、日期和考勤状态	考勤列表显示记录
学籍状态	填写状态、原因和操作日期	学籍记录保存成功
课程安排	选择班级、课程、教师和节次	课程安排列表可查询
角色权限	维护角色和权限基础信息	基础信息保存并可删除
统计报表	进入各统计页面	页面展示汇总指标和明细表

测试设计以系统主要业务流程为中心，重点验证登录、查询、新增、修改、删除

和报表展示是否能够形成完整闭环。由于课程设计系统通常在本地环境运行，测试时还需要确认 MySQL 服务是否启动、数据库名称和账号密码是否与配置文件一致、端口 81 是否被占用以及浏览器是否能够正常加载静态资源。

在功能验证过程中，若列表页面能够正确分页，说明 PageHelper 配置和 Mapper 查询能够正常协作；若新增或修改后能够返回列表并显示最新数据，说明 Controller、Service 和 Mapper 之间的数据传递正常；若删除主数据时相关业务表没有出现外键异常，说明服务层的关联删除逻辑发挥了作用。

对于角色权限模块，测试重点不是验证完整访问控制，而是验证角色和权限基础信息是否能够维护。当前代码没有提供用户角色关联、角色权限关联和统一拦截器，因此不能把该模块测试为完整 RBAC 闭环。论文中按照代码实际实现进行描述，可以避免功能夸大。

对于统计报表模块，测试时需要关注统计口径是否与数据来源一致。例如学生统计来自学生表和班级年级字段，成绩统计来自成绩表并关联学生和课程，考勤统计来自考勤状态字段，教师统计来自教师和部门信息，课程安排统计来自课程安排表。只有数据来源明确，报表结果才具有解释价值。

从实现结果看，系统已经形成较完整的后台管理流程。登录后进入首页，管理人员可以按菜单进入不同模块。列表页面提供分页和查询，编辑功能复用新增与修改逻辑，删除功能通过服务层处理主表及相关业务数据。统计报表页面则对学生、成绩、考勤、教师和课程安排进行汇总展示，使系统不仅能维护数据，也能对数据进行初步分析。

第 5 章 总结

本文围绕基于 Spring Boot 的中学生信息管理系统进行了分析与论文整理。在阅读项目源代码和数据库脚本的基础上，按照模板既定标题结构，从系统开发背景、社会需求、系统目标、开发环境、可行性分析、功能需求、数据库设计和系统实现等方面进行了说明。论文内容已从模板示例主题转为本项目实际对应的中学生信息管理业务。

从功能实现看，系统覆盖了学生信息、年级、班级、课程、成绩、课程安排、教师、部门、学籍状态、考勤、角色、权限和统计报表等模块。各模块采用较统一的控制器入口和服务层处理方式，能够完成后台管理系统常见的列表查询、新增修改、删除和分页展示。数据库结构围绕学生和教学业务展开，多个表之间通过外键或业务字段形成联系，能够满足课程设计层面的数据管理需求。

从技术实现看，系统使用 Spring Boot 简化项目搭建，使用 Spring MVC 处理请求，使用 MyBatis 完成 SQL 映射，使用 MySQL 保存业务数据，使用 Thymeleaf 展示页面。项目目录结构较清晰，控制层、服务层、持久层和页面层之间职责相对明确。统计报表模块虽然没有使用复杂图表组件，但通过 Java 集合运算和模板页面展示实现了基础数据汇总。

系统仍存在可以进一步完善的地方。首先，登录功能目前采用固定管理员账号，缺少用户表、密码加密和完整权限拦截机制；其次，角色和权限之间缺少正式关联表，RBAC 功能仍停留在基础数据维护层面；再次，部分删除逻辑通过手动处理关联数据完成，后续可以结合事务管理和更完整的数据库约束进行优化；最后，统计报表可进一步使用 ECharts 等图表组件增强可视化效果。

总体而言，本系统能够体现 Java Web 管理系统从需求分析、数据库设计到功能实现的基本过程。通过本次课程设计，能够加深对 Spring Boot 项目结构、MyBatis 映射机制、关系型数据库设计、Thymeleaf 页面渲染和后台管理系统常见业务流程的理解，也为后续开发更完整的校园信息管理平台奠定基础。

参考文献

- [1] 王珊, 杜小勇, 陈红. 数据库系统概论[M]. 第 6 版. 北京: 高等教育出版社, 2023.
- [2] 凯·S.霍斯特曼. Java 核心技术 卷 I : 基础知识[M]. 北京: 机械工业出版社, 2022.
- [3] 明日科技. Spring Boot 项目开发实战[M]. 北京: 清华大学出版社, 2022.
- [4] 杨东. 基于 Spring Boot 的学生信息管理系统的设计与实现[D]. 武汉: 武汉工程大学, 2021.
- [5] 韦彩仁, 张锐杭. 基于 Vue 3 和 Spring Boot 的学生信息管理系统设计与实现[J]. 电脑编程技巧与维护, 2024(10):3-6,20.
- [6] 谢振华. 基于 Vue.js 与 Spring Boot 的教务管理系统设计[J]. 电脑与信息技术, 2024,32(4):95-97,101.
- [7] 叶小波. 基于云计算的高校学生信息分类管理系统设计[J]. 自动化技术与应用, 2024,43(10):186-189,194.
- [8] 宋桂平. 基于高校学生管理系统的数据库管理云存储技术研究[J]. 科技创新与应用, 2024,14(19):159-162.
- [9] 王磊. MySQL 数据库技术及其应用[M]. 北京: 清华大学出版社, 2022.
- [10] 李刚. 疯狂 Java 讲义[M]. 第 5 版. 北京: 电子工业出版社, 2021.